SVEUČILIŠTE U ZAGREBU
FAKULTET STROJARSTVA I BRODOGRADNJE

ZAVRŠNI RAD

Non-sequential task execution with two industrial robots

Mentor: Student:

Assoc. prof. Marko Švaco, PhD Damjan Drobac

Zagreb, 2026.

I declare that I prepared this thesis independently, using the knowledge acquired during my studies, the listed literature and the professional guidance of my mentor. I would like to thank my mentor, assoc. prof. Marko Švaco, PhD, for his guidance, advice and support during the preparation of this thesis. I would also like to thank my colleague Matija Markulin for his help during the setup of the robots and for his useful suggestions and ideas during the problem-solving process. Finally, I would like to thank the Faculty of Mechanical Engineering and Naval Architecture for providing access to the software tools and robotic equipment used during the development and testing of the presented solution.

Damjan Drobac

SVEUČILIŠTE U ZAGREBU
FAKULTET STROJARSTVA I BRODOGRADNJE

Središnje povjerenstvo za završne i diplomske ispite
Povjerenstvo za završne ispite studija
mehatronika i robotika

Sveučilište u Zagrebu Fakultet strojarstva i brodogradnje	
Datum	Prilog
Klasa: 602-01/26-01/3	
Ur.broj: 15-31000-26-	

ZAVRŠNI ZADATAK

Student: **Damjan Drobac** JMBAG: 0035248214

Naslov rada na hrvatskom jeziku: **Nesekvencijalno izvršavanje zadataka s dva industrijska robota**

Naslov rada na engleskom jeziku: **Non-sequential task execution with two industrial robots**

Opis zadatka:

In this bachelor's thesis it is necessary to investigate and develop an approach for the non-sequential operation of two industrial robots on a shared object, i.e. a common workpiece. The task includes the development and analysis of simulation models in the RoboDK and CoppeliaSim software environments, with the aim of evaluating the feasibility of simultaneous multi-robot operation on a shared workpiece.

The capabilities of the above-mentioned simulation tools should be compared, particularly with respect to ease of modelling, motion synchronization, transfer of paths and trajectories to real robots, and the possibility of integration with real robots in the laboratory.

Additional emphasis should be placed on connecting the simulation environment with real robots via ROS2. The thesis should describe possible robot connection methods and analyse the observed communication limitations of RoboDK, CoppeliaSim, and ROS2.

The bachelor's thesis should present the developed models, the algorithms used, and the communication structure of the system. It should also analyse the advantages and limitations of individual approaches and assess their practical applicability for real cooperative robot operation in a selected case study.

The bachelor's thesis must include the literature used, as well as any external assistance received.

Zadatak zadan:
22. 4. 2026.

Datum predaje rada:
2. rok: 15. i 16. 7. 2026.
3. rok: 16. i 17. 9. 2026.

Predviđeni datumi obrane:
2. rok: 21. - 23. 7. 2026.
3. rok: 22. - 24. 9. 2026.

Zadatak zadao:
izv. prof. dr. sc. Marko Švaco

Predsjednik Povjerenstva:
izv. prof. dr. sc. Petar Ćurković

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF TECHNICAL DOCUMENTATION

No separate technical drawings or additional technical documentation were prepared as separate appendices for this thesis.

LIST OF SYMBOLS

Symbol	Unit	Description
TCP	-	Tool centre point
IK	-	Inverse kinematics
FK	-	Forward kinematics
ROS2	-	Robot Operating System 2
RTDE	-	Real-Time Data Exchange
q	rad	Vector of robot joint coordinates
x, y, z	m	Cartesian position coordinates
q_dot	rad/s	Vector of robot joint velocities
q_ddot	rad/s^2	Vector of robot joint accelerations
dt	s	Simulation or sampling time step
f	Hz	Publishing or sampling frequency
time_from_start	s	Time value assigned to a JointTrajectory point

SAŽETAK

U ovom završnom radu istraženo je nesevencijalno izvođenje zadataka s dva industrijska robota. Razvoj rada proveden je postupno: od početnog koncepta u programu RoboDK, preko CoppeliaSim simulacije s pomičnom referencom, do ROS2 povezivanja sa stvarnim Universal Robots manipulatorima. Sustav je najprije validiran u simulaciji, zatim na jednom stvarnom UR robotu, a na kraju na dva stvarna robota, UR3e i UR5e, koji izvode koordinirani zadatak.

Nakon prve funkcionalne izvedbe s forward position controllerom provedena je dodatna analiza kvalitete gibanja. Uočeno je da slanje samo trenutnih položaja zglobova može uzrokovati vibracije, posebno kada je tok naredbi rjeđi ili vremenski manje pravilan. Zbog toga su dodatno istraženi načini slanja brzina i akceleracija zglobova preko ROS2 controla.

Najbolji rezultat postignut je metodom u kojoj se CoppeliaSim putanja prvo snimi, zatim se iz snimljenih položaja, brzina i akceleracija formira potpuna JointTrajectory putanja te se ona šalje stvarnim robotima preko joint trajectory controllera. U toj metodi controller dobiva cijelu vremenski definiranu putanju unaprijed, što omogućuje glatko gibanje bez vidljive vibracije cijevi.

Ključne riječi: nesevencijalno izvođenje, industrijski roboti, CoppeliaSim, ROS2, Universal Robots, UR3e, UR5e, forward position controller, joint trajectory controller, brzine zglobova, akceleracije zglobova, sinkronizacija.

SUMMARY

This thesis investigates non-sequential task execution with two industrial robots. The work was developed gradually, starting from a RoboDK proof of concept, continuing with a CoppeliaSim simulation of a moving-reference task, and ending with a ROS2-based connection to real Universal Robots manipulators. The final laboratory setup uses a real UR3e and UR5e in a coordinated task in which one robot changes the reference situation that the other robot follows.

After the first functional simulation-to-real execution with the forward position controller, an additional motion-quality analysis was carried out. The experiments showed that sending only instantaneous joint positions can cause vibration, especially when the command stream is sparse or temporally uneven. Therefore, the possibility of using joint velocities and accelerations through ROS2 control was investigated.

The best result was achieved with a recorded JointTrajectory approach. The CoppeliaSim motion was first recorded, including joint positions, velocities and accelerations. These data were then converted into complete time-defined JointTrajectory commands and sent to the real robots through the joint trajectory controller. Since the controller received the full trajectory in advance, the resulting physical motion was smooth without any vibration.

Key words: non-sequential execution, industrial robots, CoppeliaSim, ROS2, Universal Robots, UR3e, UR5e, forward position controller, joint trajectory controller, joint velocities, joint accelerations, synchronization.

1. INTRODUCTION

1.1. Motivation, objective and scope

Industrial robots are widely used for tasks that require repeatability, accuracy and continuous operation. In many industrial applications, robots are programmed to execute predefined sequences of movements in a controlled environment. However, more demanding robotic tasks require coordination between multiple robots or between a robot and a moving object. In such cases, the task cannot be described only as a fixed sequence of independent robot programs. Instead, the motion of one robot can directly influence the reference frame, target position or path that another robot needs to follow.

The topic of this thesis is the non-sequential task execution with two industrial robots. The main idea is to investigate a situation in which two robots work on a shared object or a shared workspace, where one robot changes the position or orientation of the object or path, while the other robot follows a selected point or line related to that moving object. This type of task introduces practical challenges related to path definition, inverse kinematics, synchronization, joint stability and transfer of the simulated motion to real robotic systems.

Objective of the thesis

The objective of this thesis is to develop and compare approaches for non-sequential task execution with two industrial robots. The work combines RoboDK and CoppeliaSim modelling, motion analysis, ROS2-based simulation-to-real transfer and an experimental comparison of controller strategies on real UR robots. Particular attention is given to path definition, inverse kinematics, joint continuity and synchronization.

The specific objectives of the thesis are:

- to create an initial RoboDK simulation for point and line following with two robots,
- to record joint values of both robots and use them for replay and analysis,
- to identify problems related to inverse kinematics, joint jumps and configuration resets,
- to develop a CoppeliaSim model in which a moving path is followed by the second robot,
- to investigate the connection between CoppeliaSim, ROS2 and a real robot,
- to compare the advantages and limitations of RoboDK, CoppeliaSim and ROS2-based execution.

- to experimentally compare live and recorded controller strategies using position-only commands and time-defined JointTrajectory execution.

Scope of the work

The scope of the thesis includes simulation-based development, robot-motion analysis and practical ROS2 communication tests. RoboDK was used for point and line following and configuration analysis, CoppeliaSim for modelling the moving-reference task, and ROS2 for connecting simulated joint motion with the real robot controllers. The result is a functional experimental setup that demonstrates non-sequential robot coordination rather than a production-ready industrial cell.

1.2. Development path and thesis structure

Development began with a RoboDK proof of concept for point and line following. Recorded joint data were then used to compare initial configurations and identify joint jumps, reset events and inverse-kinematics failures. The task was subsequently transferred to CoppeliaSim, where the moving-reference concept was validated first in simulation, then with one real robot, and finally with the coordinated UR3e–UR5e system.

The overall development path, from the initial RoboDK proof of concept to the final dual-robot validation and controller analysis, is shown in Figure 1.1.

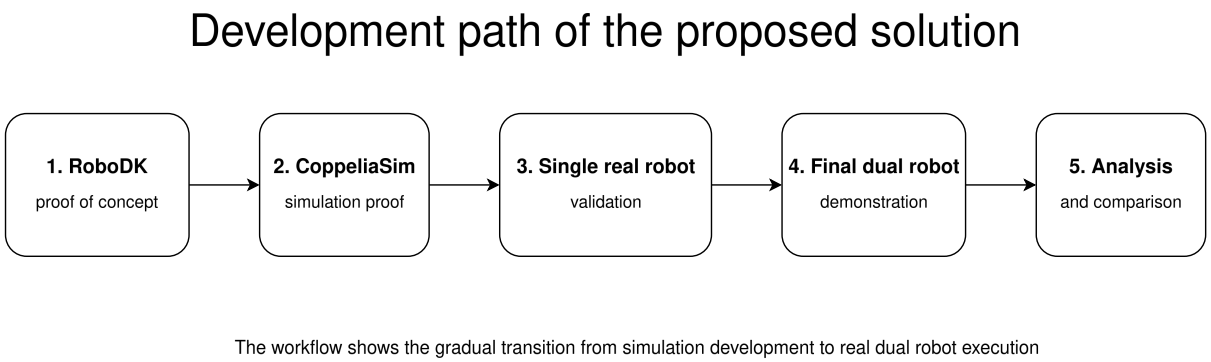


Figure 1.1. Development path of the proposed solution

Structure of the thesis

The thesis is organized into eight chapters. Chapters 1 and 2 introduce the problem and theoretical background. Chapters 3 and 4 describe the development in RoboDK and CoppeliaSim, while Chapters 5 and 6 present the ROS2 integration and final dual-robot system. Chapter 7 compares the investigated controller strategies, and Chapter 8 presents the conclusions and possible future work.

2. THEORETICAL BACKGROUND

This chapter introduces the concepts required to understand the developed system: robot kinematics, path and trajectory execution, multi-robot coordination, simulation tools and ROS2 communication. The focus is on concepts directly applied in the non-sequential task, where the motion of one robot changes the reference frame or path followed by the other robot.

2.1. Industrial manipulators and coordinate descriptions

Industrial robot manipulators are programmable mechanical systems used to move a tool or workpiece with a defined pose in space. A serial industrial robot is usually composed of a base, several links and a number of rotary or linear joints. In this thesis, the practical focus is on six-axis industrial robot manipulators, because this type of robot can position and orient a tool centre point in three-dimensional space.

The two real robots used in the later part of the thesis are Universal Robots e-Series manipulators, specifically UR3e and UR5e. These robots are collaborative industrial robots with six revolute joints. In the context of this work, they are not used for force interaction with a human operator, but as robotic platforms that can receive joint commands from a ROS2-based control system.

Robot motion can be described by Cartesian tool poses or by joint positions. In this work, target-based Cartesian poses were used in RoboDK, Cartesian targets were followed through inverse kinematics in CoppeliaSim, and joint commands were transferred to the real robots through ROS2.

Joint space and Cartesian space

Joint space describes a robot configuration by the values of its individual joints. For a six-axis robot, this is usually represented by a vector of six joint angles. Cartesian space describes the pose of the robot tool in the workspace, usually by position and orientation with respect to a selected coordinate frame.

Both descriptions were used during the thesis. In RoboDK, target poses were used to define points and line-following targets, but joint values were recorded to analyze stability and replay the motion. In CoppeliaSim, a dummy object named target2 was used as the moving path-following target. The script moved this dummy along the path in Cartesian space, while the robot joint values were calculated through inverse kinematics. In the ROS2 connection,

the practical command transferred to the real robot was a vector of joint positions or, in the recorded trajectory experiments, a complete time-defined JointTrajectory.

The reason for transferring joint positions in the final connection is practical. If a simulated robot and a real robot have the same kinematic structure and joint ordering, a joint vector from the simulation can be sent to the real robot controller through an appropriate ROS2 controller. This approach reduces the need to solve inverse kinematics on the real robot side during each control step, but it requires that the real robot starts from a synchronized configuration.

2.2. Kinematics, path following and multi-robot coordination

Forward kinematics calculates the pose of the robot tool from known joint positions. Inverse kinematics solves the opposite problem: it calculates joint positions that can achieve a desired tool pose. The distinction appears clearly in the CoppeliaSim part of the work. Robot 1 was controlled in forward-kinematic mode because the script directly assigned joint positions to the robot. Robot 2 was controlled in inverse-kinematic mode because its task was to move its tool toward the moving path-following target.

Inverse kinematics can have multiple solutions, no solution or solutions that are not desirable for a particular task. This was observed during the RoboDK development, where Robot 2 sometimes changed its configuration branch or produced sudden joint jumps during line following. Because of that, the RoboDK part included a configuration analysis in which different initial configurations were tested and compared using recorded joint data.

In the final ROS2 connection, the main challenge was not only the kinematic calculation itself, but the consistency between the simulated and real robot states. If the real robot is not aligned with the simulated robot before streaming begins, the controller can receive a command that is not compatible with the current physical state of the robot. This was the reason for introducing a synchronization step before run mode.

Path following and trajectory execution

Path following is the task of moving a robot tool along a defined geometric path. The path can be defined as a continuous curve, as a set of discrete targets or as a time series of joint positions. In the RoboDK phase, the line was represented as a sequence of generated targets. In the CoppeliaSim phase, the target was moved along a stored path using interpolated positions and orientations. In the ROS2 phase, the motion of the simulated robot was transferred to the real robot as joint commands.

Trajectory execution is more restrictive than simple path following because it includes timing. A trajectory does not only describe where the robot should move, but also when it should be at each point. This is important for multi-robot systems because two robots must remain coordinated during execution. In the thesis, a full industrial trajectory planner was not developed. Instead, the emphasis was placed on simulation-driven joint command transfer and synchronization before execution.

Multi-robot coordination

Multi-robot coordination ensures that the actions of two or more robots remain compatible in time and space. Unlike sequential execution, simultaneous coordination allows the task of one robot to depend on the motion of another. In the developed system, Robot 1 moved the reference path while Robot 2 tracked it, requiring separate topics, controllers and synchronization.

Non-sequential task execution

In this thesis, non-sequential execution means that Robot 2 does not follow an independent predefined program because its target changes with the motion of Robot 1. The concept was first evaluated in RoboDK, modelled through a moving object hierarchy in CoppeliaSim and finally transferred to real UR robots through ROS2.

2.3. Simulation tools and ROS2 communication

Robot simulation allows tasks, configurations and path feasibility to be evaluated before physical testing. However, real execution introduces controller limits, communication delays, safety checks and cycle-time requirements. Simulation was therefore used as a development tool and subsequently connected to real UR robots through ROS2.

RoboDK as an offline programming and simulation tool

RoboDK was selected for the initial proof of concept because its target-based workflow enabled rapid station setup, path definition and joint-data analysis [1]. It was suitable for offline development, but coordinated execution on two physical robot controllers required an additional communication and synchronization layer.

CoppeliaSim as a robotic simulation environment

CoppeliaSim was selected because it supports object hierarchies, embedded scripts, inverse-kinematic groups and ROS2 communication [2]. According to the official documentation, CoppeliaSim can act as a ROS2 node and communicate through topics, services and

actions [2]. Attaching the path to Robot 1 allowed its world pose to change with the robot motion, while Robot 2 used inverse kinematics to follow the moving target.

ROS2 and communication with real robots

ROS2 provided the communication layer between CoppeliaSim and the real robots [3]. ROS2 structures communication through nodes and topics, allowing independent software components to exchange data [3]. The Universal Robots ROS2 driver connected ROS2 to the UR3e and UR5e controllers [4], while the External Control URCap enabled the robot controllers to accept commands from the external computer [5].

The forward position controller was initially selected because it accepts target joint positions directly [6] through a Float64MultiArray command topic [7]. Together, CoppeliaSim, ROS2, the UR driver and the controller formed the simulation-to-real connection described in Chapter 5.

3. INITIAL APPROACH IN ROBODK

The RoboDK phase had two objectives: to demonstrate point and line following on a moving object and to identify problems caused by inverse kinematics, configuration changes and joint discontinuities. The work progressed from point tracking to line following, joint recording, replay and configuration analysis, providing the basis for the later CoppeliaSim and ROS2 development.

3.1. RoboDK station and proof-of-concept setup

RoboDK was selected for the initial proof of concept because its Python API supports rapid station setup and target-based robot control. The station contained two industrial robot models: Robot 1 moved the reference object, while Robot 2 followed a point and later a line attached to that object. This setup was used to evaluate the basic task before introducing communication with physical robots.

The initial RoboDK station and the roles of the two robots are shown in Figure 3.1.

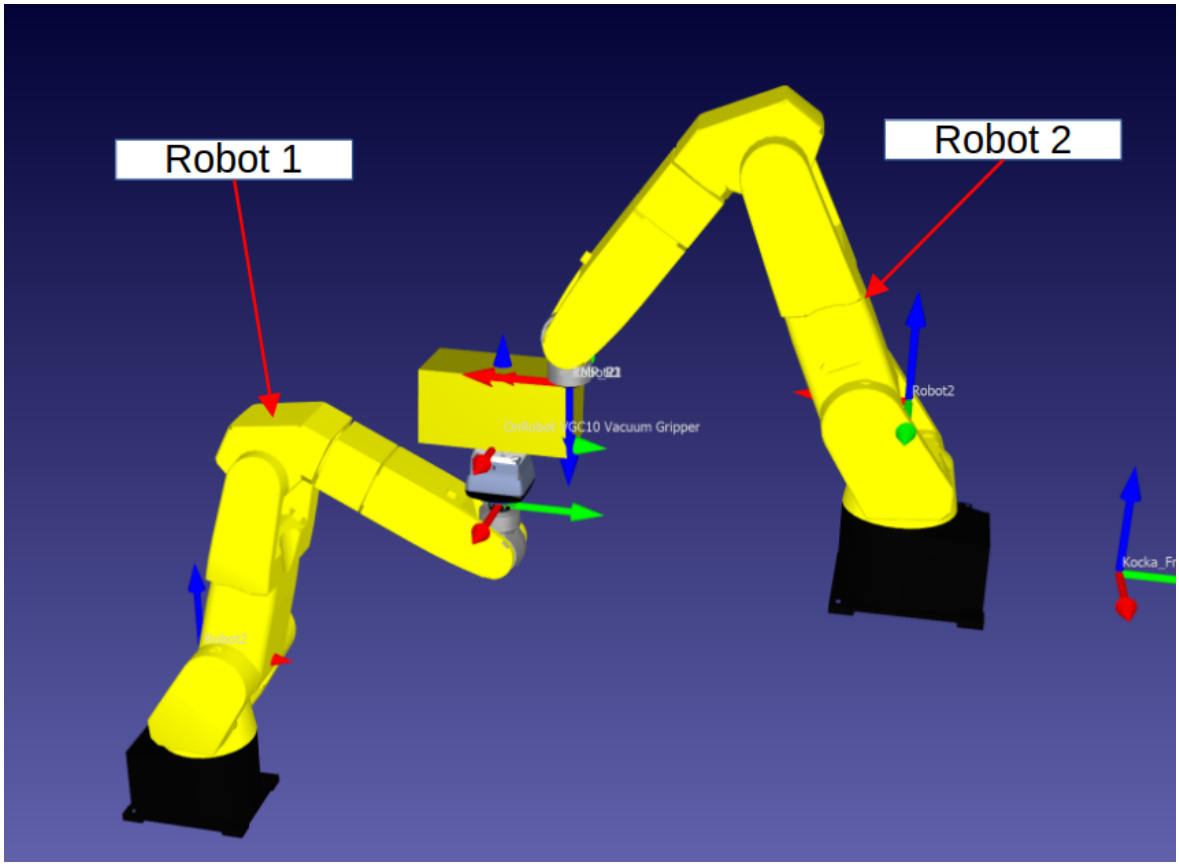


Figure 3.1. RoboDK line-following station with two robots and a moving workpiece

Initial RoboDK station setup

The station contained two robot models, a moving workpiece and targets for point- and line-following tests. Robot 1 executed a predefined program, while Python scripts controlled Robot 2 and tested different initial configurations. Because the targets moved with Robot 1, Robot 2 tracked changing world poses rather than a fixed global target, making the task non-sequential.

The organization of the RoboDK station, including the generated line targets and the tested initial configurations, is shown in Figure 3.2.

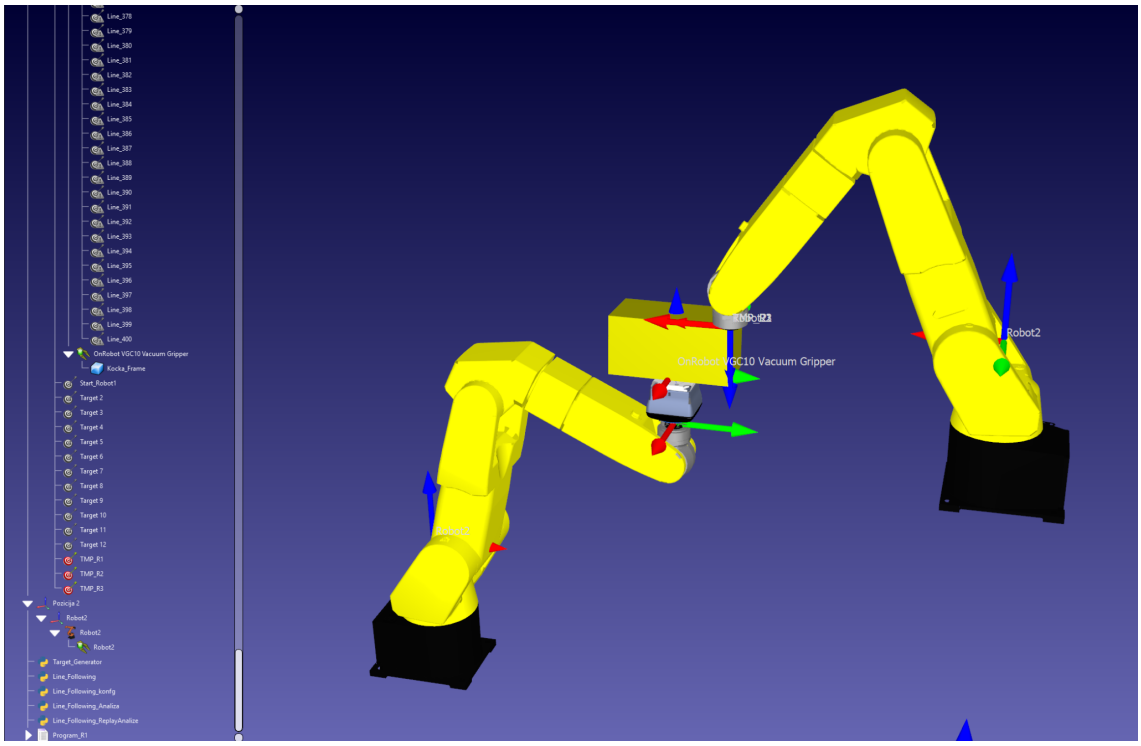


Figure 3.2. RoboDK station tree with line and TMP targets

The complete RoboDK procedure is summarized in Figure 3.3. The diagram shows how the initial station setup was extended from point tracking to line following, recorded joint-data analysis and the selection of the most stable configuration before the work was transferred to CoppeliaSim.

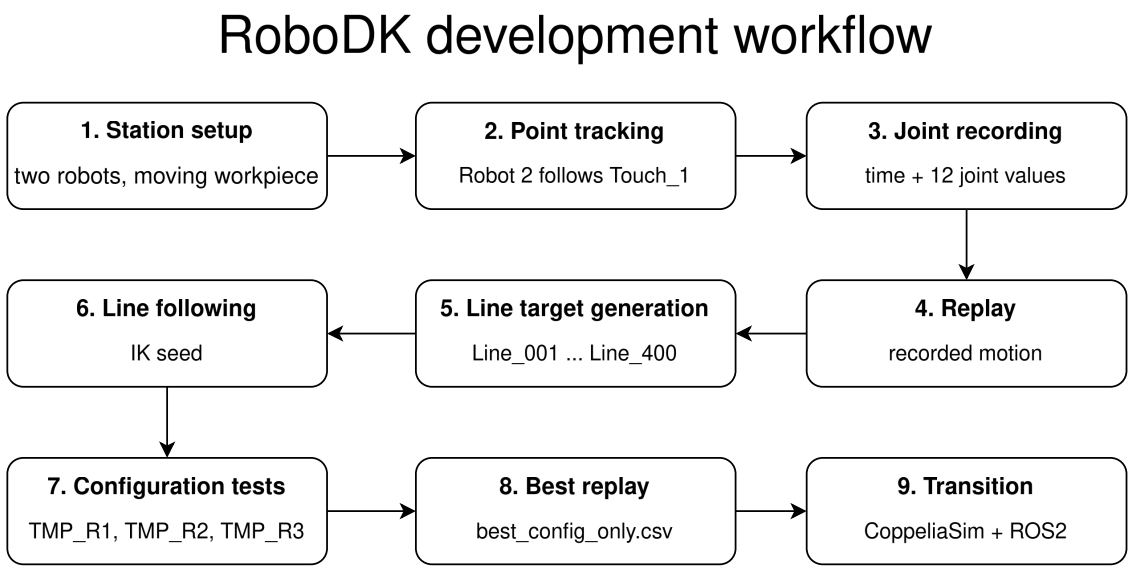


Figure 3.3. RoboDK development flow diagram

The main scripts used during the RoboDK development and their roles are listed in Table 3.1.

Table 3.1. Main RoboDK scripts used during the initial development phase

Script	Purpose	Main output or role
Dot_Following.py	Robot 2 repeatedly follows the target Touch_1 while Program_R1 is running.	Initial proof of point tracking
Dot_Following_Record.py	Records time and the six joint values of both robots during point tracking.	R1R2_Record_*.csv
Dot_Following_Replay.py	Replays both robots using the recorded joint values from the newest CSV file.	Simulation replay from recorded joints
Target_Generator.py	Generates or updates line targets between selected corner targets.	Line targets used for line following
Line_Following.py	Robot 2 follows the ordered line targets while Robot 1 executes its program.	Line-following simulation
Line_Following_konfg.py	Automatically tests multiple initial configurations using TMP_Ri targets.	configs_line_following.csv
Line_Following_Analysis.py	Calculates motion quality metrics and selects the best configuration.	analysis_summary.csv and best_config_only.csv
Line_Following_Replay_Analysis.py	Replays the selected best configuration using recorded joint values.	Smooth replay of best configuration

Point tracking experiment

The first experiment tested whether Robot 2 could follow the moving target Touch_1 while Robot 1 executed Program_R1. Robot 2 was first moved to Touch_1 using a blocking joint motion. During Robot 1 motion, non-blocking MoveJ commands were updated every 0.005 s, corresponding to approximately 200 Hz and producing visually smooth tracking in simulation.

Pseudocode 3.1. RoboDK point-tracking procedure

```
INPUT:
    Moving target
    Predefined motion of Robot 1
    Update period

Move Robot 2 to the initial target pose
Start the predefined motion of Robot 1

WHILE Robot 1 is moving DO
    Obtain the current pose of the target attached to the moving workpiece
    Command Robot 2 toward the current target pose without blocking execution
    Wait until the next update instant
END WHILE
```

The experiment confirmed that Robot 2 could follow a target whose world pose changed with Robot 1. However, it remained a simulation-level method based on frequent target updates and did not address synchronization between physical robot controllers.

3.2. Recording, replay and line-following development

After point tracking, the motion of both robots was recorded to separate motion generation from later analysis and replay. During each loop iteration, the script stored the elapsed time and six joint values from each robot in a file named according to the pattern R1R2_Record_YYYYMMDD_HHMMSS.csv.

Each sample therefore contained one timestamp and twelve joint positions, providing a time-dependent description of the complete simulated system. The recorded data were subsequently used for replay and configuration analysis.

Replay based on recorded joint values

A separate replay script searched for the newest R1R2_Record_*.csv file and reproduced the motion by applying the stored joint positions at their recorded timestamps. During replay, the corresponding values were applied to both robots when each recorded time was reached.

This approach made the motion repeatable and removed the need to solve inverse kinematics again during replay. Although direct joint assignment through setJoints() is straightforward in

simulation, equivalent execution on two physical controllers requires explicit command timing and synchronization. This limitation motivated the later transition to CoppeliaSim and ROS2.

Pseudocode 3.2. Replay based on recorded joint values

```
INPUT:
    Recorded timestamps
    Recorded joint positions of Robot 1 and Robot 2

Load the recorded dataset
Set the replay time origin

FOR each recorded sample DO
    Wait until the elapsed replay time reaches the sample timestamp
    Apply the recorded joint positions to Robot 1
    Apply the recorded joint positions to Robot 2
END FOR
```

Transition from point tracking to line following

After point tracking had been validated, the task was extended to a line attached to a rectangular workpiece. Robot 1 moved and rotated the workpiece, while Robot 2 followed an ordered sequence of target poses defined along one of its edges. Since the targets moved with the workpiece, Robot 2 had to progress along the line while continuously adapting to their changing absolute poses.

Compared with single-point tracking, this introduced a stronger dependence on inverse kinematics, the initial robot configuration and joint-motion continuity. The line was represented as a discrete sequence of targets named Line_001, Line_002 and so on, with the complete program supporting targets up to Line_400. This representation made the path easier to inspect, modify, record and analyze.

Target generation for line following

A target-generation script was created to avoid preparing the line manually. It read the existing Angle_4 and Beginning targets and generated 100 intermediate targets for one line segment, named Line_301 to Line_400, following the naming structure expected by the line-following program.

The script compared the displacement along the X, Y and Z axes and selected the axis with the largest absolute change. Target positions were generated by linear interpolation along that axis, while the orientation was retained from the start target. Existing targets were updated instead of duplicated.

Pseudocode 3.3. Generation of line-following targets

```
INPUT:
    Start pose
    End pose
    Required number of targets

Read the start and end poses
Calculate the displacement along the X, Y and Z axes
Select the axis with the largest absolute displacement

FOR each required target DO
    Calculate the interpolation factor
    Copy the start pose
    Interpolate the position along the selected axis
    Keep the orientation of the start pose

    IF a target with the required identifier already exists THEN
        Update its pose
    ELSE
        Create a new target
    END IF
END FOR
```

This procedure allowed the line segment and target density to be regenerated consistently without rebuilding the station manually.

Line-following control logic

The line-following script controlled Robot 2 while Robot 1 executed its predefined motion. At initialization, it loaded the ordered target sequence, motion parameters and fallback target. For each target, the script read its absolute position and orientation, transformed the pose using the active robot frame and tool, and solved inverse kinematics using the previous joint configuration as the seed to favour a solution close to the current configuration.

Early tests showed that Robot 2 could rotate unexpectedly or switch configuration branches. Continuity checks were therefore used to reject abrupt joint changes. Valid solutions were applied with setJoints() and retained as the next IK seed. If IK failed or produced an invalid solution, the temporary target was updated and a non-blocking MoveJ command provided fallback motion instead of terminating execution.

Pseudocode 3.4. RoboDK line-following procedure

```
INPUT:
    Ordered target sequence
    Initial configuration of Robot 2

Store the current joint configuration of Robot 2 as the IK seed
Start the predefined motion of Robot 1
Set the current target to the first target in the sequence

WHILE Robot 1 is moving AND unprocessed targets remain DO
    Read the absolute pose of the current target
    Transform the target pose into the coordinate frame of Robot 2
    Solve inverse kinematics using the previous joint configuration as the seed

    IF the IK solution is valid AND the joint change is continuous THEN
        Apply the calculated joint configuration to Robot 2
        Store the configuration as the new IK seed
        Advance to the next target
    ELSE
        Update the fallback target to the required pose
        Send a non-blocking motion command toward the fallback target
    END IF
END WHILE
```

3.3. Configuration analysis and selected replay

During line-following tests, Robot 2 sometimes switched between inverse-kinematic branches, causing unexpected rotations and abrupt joint changes even when the TCP remained close to the target. Such discontinuities could not be transferred to a physical robot because they would require unrealistically high joint velocities. Joint continuity was therefore treated as a principal evaluation criterion.

To improve stability, the previous joint configuration was used as the IK seed, continuity checks were applied to reject large changes, and a temporary target provided fallback motion. These measures improved the simulation, but the result still depended on the initial robot configuration, which motivated a systematic configuration analysis.

Analysis of initial robot configurations

To evaluate the initial Robot 2 configurations systematically, Line_Following_konfg.py automatically searched for the available TMP_Ri targets. For each configuration, Robot 1 was reset to Start_Robot1, while Robot 2 was moved to Start_Robot2 through the selected TMP_Ri target. Program_R1 and the line-following procedure were then executed under the same conditions.

During execution, the script recorded the configuration index, temporary-target name, sample and target information, IK status and six joint values from each robot in

configs_line_following_detailed_notime.csv. Three configurations, TMP_R1 to TMP_R3, were tested. Each produced 1203 samples, resulting in 3609 samples in total.

A separate Python script grouped the samples by configuration and calculated the number of reset events and IK failures, the maximum single-joint and total joint jumps, the 95th percentile of the total jump and the average total jump. A weighted score was also calculated, with the largest penalty assigned to reset events, followed by IK failures and joint-jump magnitude.

Reset detection was based on angular differences between consecutive Robot 2 samples. A reset was identified when an individual joint changed by at least 120 degrees or when the sum of all joint changes exceeded 260 degrees.

The calculated motion-quality metrics for the three tested initial configurations are summarized in Table 3.2.

Table 3.2. Summary of the RoboDK configuration analysis

Configuration	Samples	Reset events	IK fail count	Max joint jump [deg]	Max total jump [deg]	P95 total jump [deg]	Average total jump [deg]
TMP_R1	1203	0	1203	5.311	12.231	3.6798	1.1308
TMP_R2	1203	2	1203	174.537	376.473	3.6016	1.7003
TMP_R3	1203	2	1203	175.690	377.558	3.8742	1.7734

The results show that TMP_R1 was the most stable tested configuration. It had zero reset events, while TMP_R2 and TMP_R3 each had two reset events. TMP_R1 also had a much smaller maximum single-joint jump and maximum total joint jump. Although the IK fail count was the same for all configurations in the recorded dataset and therefore had to be interpreted carefully, the reset-event and joint-jump metrics clearly separated the stable configuration from the less stable ones.

Based on these results, TMP_R1 was selected as the best configuration. The complete analysis was stored in analysis_summary.csv, while its samples were exported to best_config_only.csv for subsequent replay.

Replay of the selected configuration

The selected configuration was replayed from best_config_only.csv. Unlike the initial point-tracking replay, this dataset contained the line-following samples of the selected configuration. The samples were ordered by index and their joint values were applied to both robots.

To improve visual smoothness, consecutive samples were interpolated using angular wrap-around, avoiding an artificially large difference near the −180°/+180° boundary. The number of interpolation substeps and playback speed remained adjustable. The replay reproduced the selected motion stably and repeatably in RoboDK, but remained a simulation method rather than synchronized execution on two physical robot controllers.

Pseudocode 3.5. Replay of the selected robot configuration

```
INPUT:
    Recorded samples of the selected configuration
    Number of interpolation steps
    Replay speed

Load the recorded samples
Order the samples by their sample index

FOR each pair of consecutive samples DO
    Calculate the shortest angular displacement between both samples

    FOR each interpolation step DO
        Interpolate the joint positions of Robot 1
        Interpolate the joint positions of Robot 2
        Apply the interpolated joint positions to both robots
        Wait for the required replay interval
    END FOR
END FOR
```

3.4. Advantages and limitations of the RoboDK approach

The RoboDK phase confirmed that the proposed non-sequential task could be represented through point tracking and line following. It also enabled joint-data recording and the comparison of different initial configurations, which showed that the selected configuration strongly influenced motion continuity and repeatability.

The main limitation was that direct simulation commands and joint-value replay did not provide a suitable architecture for synchronized execution on two physical robot controllers. This limitation motivated the transition to CoppeliaSim and ROS2, where the moving reference frame and the simulation-to-real communication could be implemented more directly.

4. TRANSITION TO COPPELIASIM

After the initial RoboDK development and configuration analysis, the same task concept was transferred to CoppeliaSim. Before creating the final UR3e and UR5e laboratory scene, a simplified proof of concept was developed using robot models available in the simulator. Robot 1 generated motion and carried the path structure, while Robot 2 followed a moving target on that path using inverse kinematics.

After the path, moving target, scene hierarchy and IK logic had been validated, the approach was transferred to Universal Robots models and later extended to the real UR3e and UR5e setup. The UR scene therefore represented a continuation of the same method, adapted to the robot kinematics, object names, ROS2 communication and controller requirements.

The initial CoppeliaSim proof-of-concept scene and the roles of the two robots are shown in Figure 4.1.

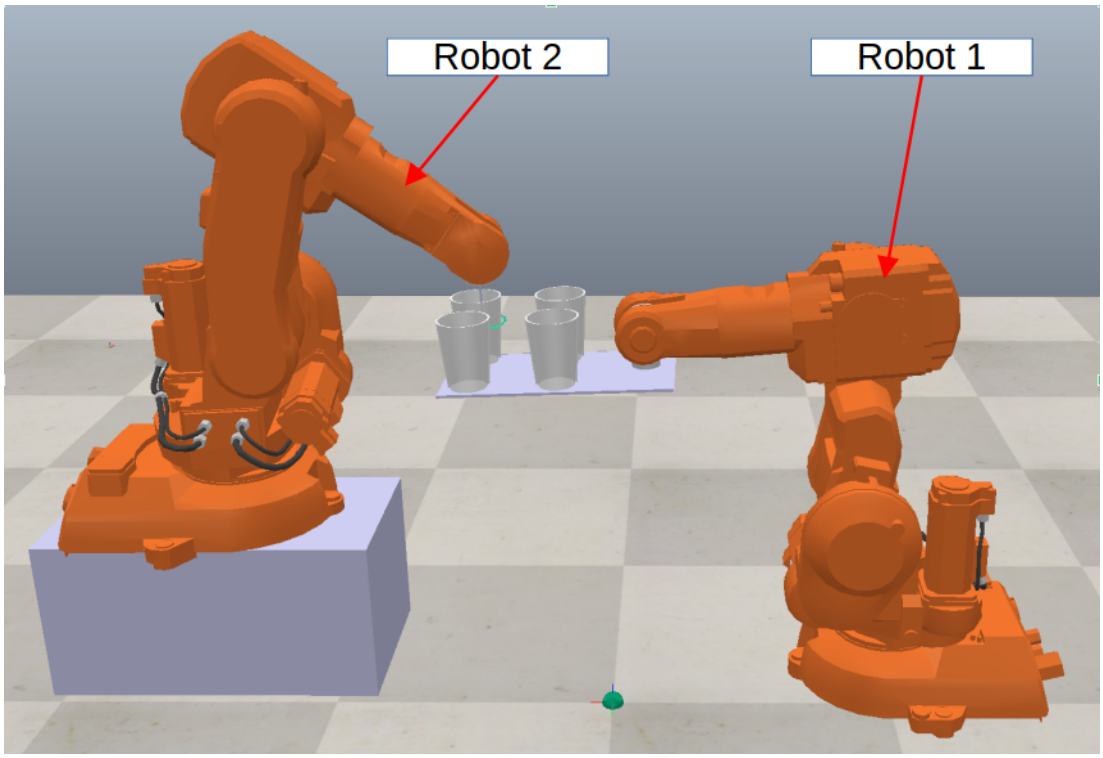


Figure 4.1. Initial CoppeliaSim proof-of-concept scene

4.1. Motivation and initial CoppeliaSim scene

The transition to CoppeliaSim was motivated by the workarounds required during the RoboDK phase. RoboDK was suitable for offline programming, target generation, joint recording and replay, but the moving line had to be represented by many individual targets.

Inverse kinematics was called from an external Python script, and additional logic was required to reduce configuration changes and joint jumps.

Although RoboDK also supports robot simulation, target handling and inverse-kinematic calculations, CoppeliaSim was selected because the moving reference frame, embedded Lua scripts, simIK-based tracking and later ROS2 communication could be integrated directly within the same simulation scene. This provided a more convenient structure for the next development phase. These features were especially suitable for this task. A path could be placed as a child object of the moving robot structure, and a target could be moved along the path during simulation. The second robot could then be controlled by an IK group that continuously attempted to bring the robot tip to the target. This allowed the motion of the path and the motion of the tracking robot to be simulated at the same time, without manually generating hundreds of individual target objects as in the RoboDK approach.

The CoppeliaSim simulation was also useful for identifying reachability limits. When the path was positioned in such a way that Robot 2 could not physically reach the moving target, the robot did not follow immediately. Instead, it effectively waited until the target entered a reachable and kinematically feasible region. This behaviour was important because it showed the practical limits of the non-sequential task. The simulation did not only show that the target could move; it also showed when the path-following task became impossible because of workspace and inverse-kinematic constraints.

Limitations observed in the RoboDK approach

The main limitation of the RoboDK approach was the representation of the line as a sequence of discrete Cartesian targets. This was sufficient for concept validation and joint-behaviour analysis, but less suitable for a continuously moving path. Since each target was evaluated individually, the resulting motion remained sensitive to the selected inverse-kinematic branch and the previous robot configuration.

Direct joint-value replay was also straightforward in simulation but could not be transferred directly to two physical robot controllers without accounting for communication timing, controller interfaces and safety constraints. The CoppeliaSim phase therefore shifted the focus from target generation and CSV replay to scene hierarchy, moving reference frames, inverse-kinematic tracking and communication with external control layers.

Initial CoppeliaSim scene

The initial CoppeliaSim scene used robot models available in the simulator to validate the moving-reference concept before introducing the final UR setup. Robot 1 generated motion in forward-kinematic mode and carried the path structure, while Robot 2 used inverse kinematics to follow the moving target. These roles were retained in the later UR scene, where synchronization, ROS2 publishers and real-robot command transfer were added.

The main elements of the CoppeliaSim scene and their roles in the moving-reference simulation are listed in Table 4.1.

Table 4.1. Main elements of the CoppeliaSim non-sequential simulation

Element	Role in the simulation	Reason for use
Robot 1	Generates periodic motion in FK mode	Moves the reference structure and therefore changes the path position in space
Robot 2	Tracks the moving target in IK mode	Represents the robot that follows the path affected by Robot 1
Path model	Stores the path geometry and orientation data	Defines the line that the target follows
Moving target dummy (target2)	Moving reference target for the path-following robot	Provides the position and orientation constraint for inverse kinematics
simIK group	Solves the IK task for Robot 2	Continuously updates Robot 2 according to the moving target

The interaction between the motion-generating robot, the path script and the path-following robot is illustrated in Figure 4.2.

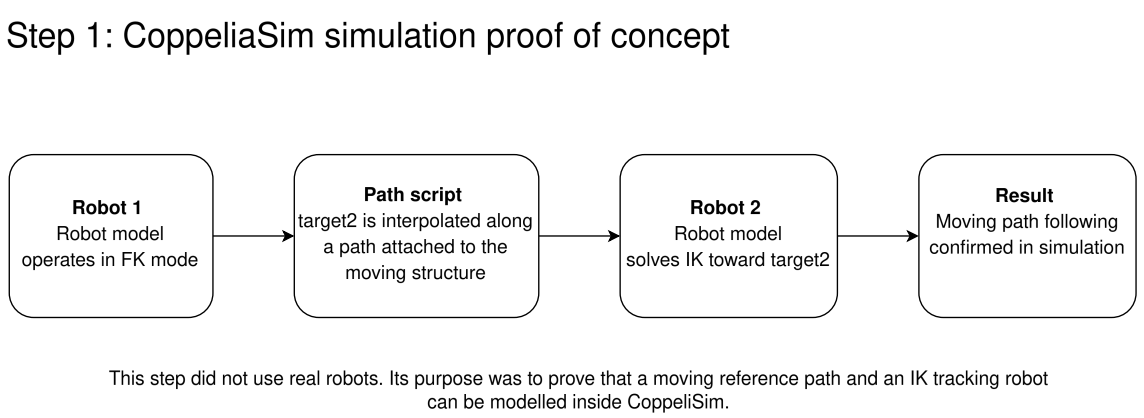


Figure 4.2. Moving-path tracking workflow in the first CoppeliaSim proof-of-concept

4.2. Moving reference frame and script implementation

The key modelling decision was to place the path and its moving target within the hierarchy of Robot 1. Since the transformation of a child object depends on its parent, the target was no longer static in the world coordinate system.

The path script moved the target along the local path coordinates while Robot 1 moved the parent structure. The target world pose therefore combined its local motion along the path with the global motion generated by Robot 1. Robot 2 followed this changing pose through inverse kinematics. Unlike fixed-path tracking, the trajectory itself was influenced by another robot, which created the required non-sequential behaviour.

Before simulation, the path script loaded the stored position and orientation data used to move the target. The same principle was later reused in the UR scene, where the path was adapted to the final circular motion around the UR3e arm.

Pseudocode 4.1. Update of the moving path target in CoppeliaSim

```
INITIALIZE:
    Load the stored path positions and orientations
    Calculate the cumulative path lengths
    Obtain the moving target object

AT EACH SIMULATION STEP:
    Calculate the travelled distance from the current simulation time
    Wrap the distance when the end of the path is reached
    Interpolate the target position along the path
    Interpolate the target orientation along the path
    Apply the interpolated pose relative to the path reference
```

Robot 1 as the motion generator

Unlike the RoboDK implementation, where Robot 1 motion was started through an external Python script, the CoppeliaSim motion was generated directly by an embedded Lua script. The script assigned periodic positions to selected robot joints, while the attached path structure moved automatically with Robot 1 through the object hierarchy.

Pseudocode 4.2. Motion generation for Robot 1 in CoppeliaSim

```
INITIALIZE:
    Obtain the controlled robot joints
    Define the start delay
    Define the maximum speed and acceleration
    Set the initial motion phase

AT EACH SIMULATION STEP:
    Calculate the elapsed time after the start delay

    IF the start delay has elapsed THEN
        Increase the motion speed up to the defined maximum
        Update the motion phase
        Calculate periodic target positions for the selected joints
        Apply the calculated joint positions to Robot 1
    END IF
```

Robot 2 as the path-following robot

In RoboDK, Robot 2 was controlled from an external Python script and followed a sequence of generated Cartesian targets. In CoppeliaSim, the tracking calculation was performed internally through a simIK group. During each simulation step, simIK calculated the robot joint configuration required to align the robot tip with the continuously moving target dummy.

Pseudocode 4.3. IK-based path following for Robot 2

```
INITIALIZE:
    Create an inverse-kinematics environment
    Create an inverse-kinematics group
    Register the robot base, tool tip and moving target
    Define the required position and orientation constraints

AT EACH SIMULATION STEP:
    Read the current pose of the moving target
    Solve the inverse-kinematics problem
    Apply the calculated joint configuration to Robot 2
    Synchronize the calculated configuration with the simulated robot model
```

The main difference from the RoboDK approach was therefore not the task itself, but the implementation structure. CoppeliaSim combined the moving object hierarchy, embedded Lua scripts and simIK tracking inside one simulation environment, whereas the RoboDK implementation relied primarily on external Python scripts and a discrete target sequence.

4.3. Practical scene setup and UR model preparation

A principal practical issue was placing the path within the reachable workspace of Robot 2. When the path was too distant or Robot 1 moved it into an unfavourable region, the IK problem became temporarily infeasible, appearing as a delay or temporary lack of motion of the following robot. The selected IK constraints functioned correctly, so the main factors were path placement, robot reachability and parent-child transformations.

The object hierarchy was essential because attaching the path to Robot 1 caused its world pose to change automatically with the robot motion. An independent path object would have remained static and would not have represented the intended task. The tests therefore showed that successful non-sequential execution depends on the path position, robot workspace, scene hierarchy and motion of the other robot.

Preparation of robot models for CoppeliaSim

After the initial CoppeliaSim proof of concept, additional work was directed toward preparing Universal Robots models that represented the real laboratory robots used in the practical part of the thesis. The simulation was gradually transferred from the generic scene to Universal Robots models, specifically UR3e and UR5e. This step was necessary before the same logic could be connected to ROS2 and the real robots.

The preparation of Universal Robots models in CoppeliaSim was based on importing URDF descriptions directly into the simulator. After import, the robot structure had to be checked inside CoppeliaSim. This required more manual work than inserting a robot from a prepared library, because the model hierarchy, joint definitions, TCP objects and mesh paths had to be verified. A specific issue appeared when the UR5e URDF used mesh paths of the form package://ur_description/..., which CoppeliaSim did not resolve automatically. The practical solution was to generate the URDF from the ROS2 xacro description and replace the package paths with absolute file paths, for example file:///opt/ros/humble/share/ur_description. After this correction, the UR5e model was imported with visible meshes and a usable joint structure.

This model-preparation step also showed an important practical difference between RoboDK and CoppeliaSim. RoboDK provides an online library with a large selection of industrial robot models that can be inserted directly into a station, which is convenient during early simulation work. In CoppeliaSim, the UR models used in this thesis had to be imported and adjusted manually before they could be used for the simulation and later ROS2 integration. CoppeliaSim was therefore more flexible for scene hierarchy, custom scripts and ROS2 communication, but it required more preparation before the UR robots behaved correctly in the simulation.

The connection to real Universal Robots also depended on the controller configuration, network setup, External Control program and active ROS2 controller. UR3e and UR5e were selected because they were supported by the Universal Robots ROS2 driver and the available

External Control workflow. These requirements influenced the later CoppeliaSim–ROS2–UR communication architecture.

The CoppeliaSim development therefore had two stages. The first validated the non-sequential path-following concept using available simulator models, while the second introduced UR models and the requirements for real execution, including the robot hierarchy, TCP selection, synchronization modes, ROS2 topics and bridge scripts. This provided the link between the initial RoboDK work and the later ROS2 connection to the real robots.

5. CONNECTION WITH THE REAL ROBOT USING ROS2

After the non-sequential motion principle had been demonstrated in the CoppeliaSim simulation, the next objective was to connect the simulated robot motion to a real Universal Robots manipulator. This chapter describes the connection between CoppeliaSim, ROS2 and the real UR robots used in the laboratory. The first real-robot validation was intentionally performed with one real UR3e robot: in the CoppeliaSim scene one simulated robot moved the path reference, while the second simulated robot followed the moving path, and only the motion of the path-following simulated robot was transferred to the real UR3e. After this single-real-robot validation, the same communication principle was extended to two real robots.

The practical result of this phase was a validated simulation-to-real chain. First, the real UR3e followed the motion of the simulated robot that was tracking the moving path. This confirmed that the CoppeliaSim-to-ROS2-to-UR command chain was functional. The final setup then applied the same idea to two robots operating at the same time, with separated ROS2 namespaces, separated ports and one bridge script per robot.

5.1. ROS2 integration and communication architecture

The transition from simulation-only work to ROS2 integration was motivated by the limitations observed in RoboDK and by the need to control real robots from the developed simulation logic. In a pure simulation, the robot state can be changed directly by calling functions such as setJoints or by solving inverse kinematics inside the simulator. A real robot, however, is controlled by its own controller and accepts motion commands only through defined communication interfaces and safety procedures.

ROS2 was introduced as the communication layer because it allows data to be exchanged between independent software components. In the developed system, CoppeliaSim provided the simulated robot motion, a Python bridge transferred the relevant joint values into ROS2 messages, and the Universal Robots ROS2 driver forwarded the commands to the real robots.

The purpose of the integration was not to replace the robot controller, but to use the simulation as the source of target joint motion. The real robots remained controlled by their UR controllers and by the External Control program on the teach pendant. ROS2 served as the interface between the simulation and the robot driver.

Communication architecture

The developed architecture consisted of four main layers: the CoppeliaSim simulation, the ROS2 communication layer, the Universal Robots ROS2 driver and the real UR robots. The simulated robots generated joint motion. The bridge read or received these joint values and published them to ROS2 topics. The active ROS2 controllers accepted the joint commands and sent them to the corresponding real robots through the UR driver.

The communication architecture connecting CoppeliaSim, ROS2, the Universal Robots ROS2 driver and the physical robots is shown in Figure 5.1.

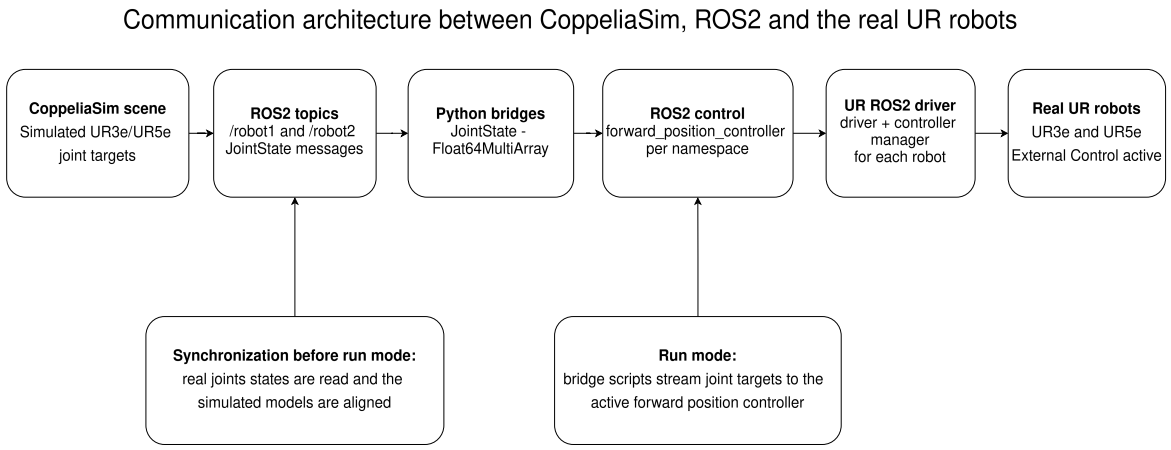


Figure 5.1. Communication architecture of the ROS2-based setup

For the dual-robot setup, each robot required its own logical separation. The ROS2 side used separated namespaces and topics for Robot 1 and Robot 2. This prevented commands, joint states and controller interfaces from being mixed between the two robots. In addition, separate network interfaces and port sets were used so that both robots could be connected at the same time without reverse-port conflicts.

5.2. UR driver, External Control and network configuration

The real robots were connected using the Universal Robots ROS2 driver. The driver provides the ROS2 interface for Universal Robots and is launched with the robot type and the robot IP address. For example, the robot type parameter is set to ur3e or ur5e depending on the robot that is connected. The official documentation describes the driver as the ROS2 package used for Universal Robots CB3 and e-Series robots [4].

On the robot side, the External Control URCap is required. This is the component that allows the robot controller to receive commands from an external PC. The official Universal Robots

documentation states that the External Control URCap is needed when using the client library with a real robot, because it enables external control from a remote PC [5].

In the practical setup, the External Control program was opened and started on the teach pendant. The host IP address corresponded to the IP address of the computer network interface connected to the robot. The port value had to match the script sender port used by the ROS2 driver. This connection between the teach pendant settings and the launch parameters was essential, especially when both robots were used at the same time.

The External Control configuration used on the teach pendant is shown in Figure 5.2.

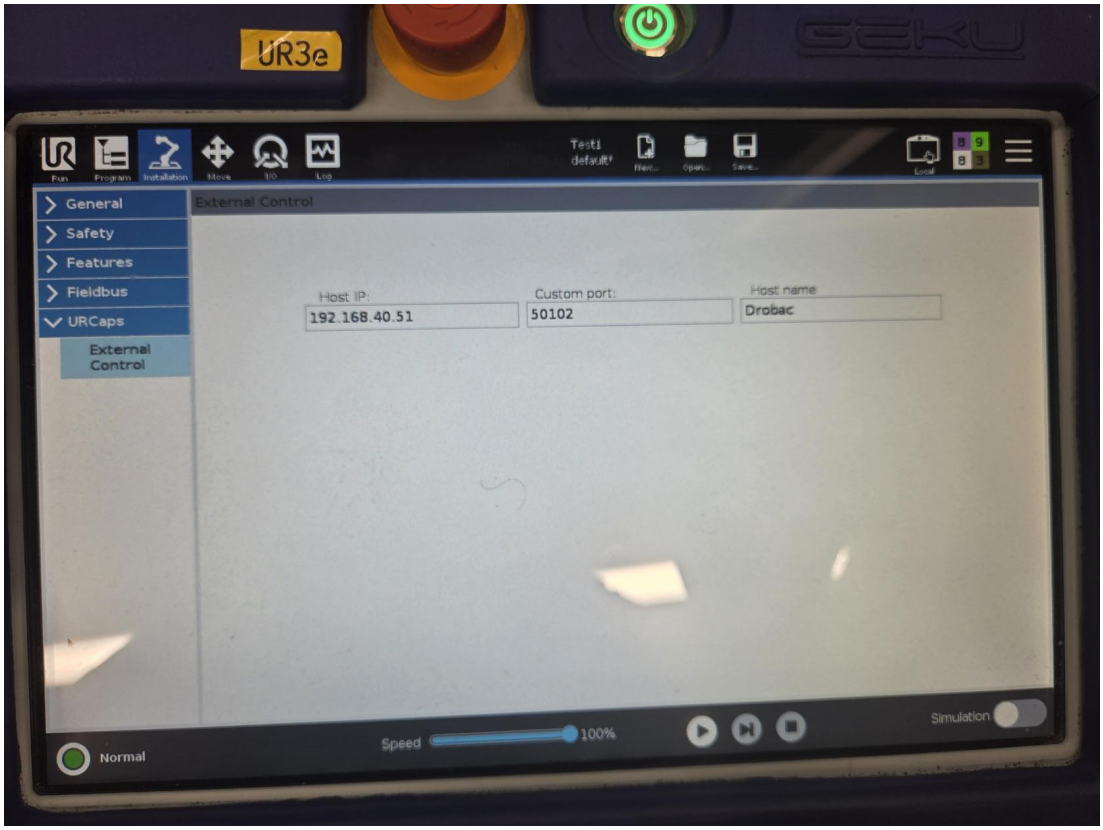


Figure 5.2. External Control setup on the Universal Robots teach pendant

Network configuration and robot connection

A stable network configuration was required before the robots could be controlled through ROS2. Both robots were connected to the same laboratory network and used a common ROS2 domain. Each robot was assigned a separate network address, ROS2 namespace and non-conflicting communication port set, while both teach pendants communicated with the same control computer.

Non-conflicting communication ports were required for simultaneous operation, because two driver instances could not use the same reverse communication port.

Code listing 5.1. Canonical start-up command for the namespaced dual-robot UR driver

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

ros2 launch ~/ur_ns/dual_ns.launch.py
```

Code listing 5.2. Verification of joint-state topics after starting the driver and External Control

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

ros2 topic list | grep joint_states

# expected topics
/robot1/joint_states
/robot2/joint_states
```

5.3. Controllers, bridge and synchronization

The UR ROS2 driver is integrated with ros2_control. This means that robot motion is not sent directly to the hardware from an arbitrary script. Instead, commands are sent through controllers that expose specific ROS2 interfaces. During the work, several controllers were tested. The first functional simulation-to-real transfer was achieved with the forward position controller, while additional experiments investigated whether smoother motion could be obtained by sending time-defined joint trajectories that include positions, velocities and accelerations.

Several ROS2 control interfaces were investigated during the work. The Universal Robots ROS2 driver documentation describes the controllers available for robot command and state interfaces [6]. The main comparison was performed between forward_position_controller and joint_trajectory_controller. The forward position controller receives target joint positions through a Float64MultiArray command message [7]. The joint trajectory controller executes time-defined trajectories through the joint_trajectory_controller interface [8], while the structure of the JointTrajectory message, including joint positions, velocities, accelerations and time values, is defined in [9]. A live streaming variant based on the FollowJointTrajectory action interface was also tested [10], but it was not selected as the final method because repeatedly replacing short action goals did not produce stable continuous tracking.

The roles of the ROS2 controllers considered during the experimental work are summarized in Table 5.1.

Table 5.1. Controllers relevant to the developed ROS2 connection

Controller	Role in the experimental work
joint_state_broadcaster	Publishes measured joint states from the real robots; used for synchronization and graph analysis.
forward_position_controller	Receives direct joint position commands as Float64MultiArray. Used as the initial live baseline and recorded position replay method.
forward_velocity_controller	Receives joint velocity commands. It was investigated as a possible live method, but it does not contain the full future path and was not selected as the final smooth method.
joint_trajectory_controller	Receives time-defined JointTrajectory data containing joint positions, velocities, accelerations and time_from_start values. It produced the smoothest result when the full recorded trajectory was sent at once.

Switching to the forward position controller

Before the real robot could follow joint values streamed from the simulation, the controller required for the current experiment had to be activated. The controller that was active at that moment was deactivated, and the required controller was activated separately for each robot namespace. This principle was used both for forward_position_controller experiments and for recorded joint_trajectory_controller execution.

Code listing 5.3. Controller switching and verification commands

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

ros2 control switch_controllers \
  --controller-manager /robot1/controller_manager \
  --deactivate <currently_active_controller> \
  --activate forward_position_controller

ros2 control list_controllers \
  --controller-manager /robot1/controller_manager

ros2 control switch_controllers \
  --controller-manager /robot2/controller_manager \
  --deactivate <currently_active_controller> \
  --activate forward_position_controller

ros2 control list_controllers \
  --controller-manager /robot2/controller_manager
```

When two robots were used, the same principle had to be applied separately for both robot namespaces. Each robot had its own controller manager and its own forward position command topic. This separation was necessary because otherwise one robot could overwrite or interfere with the control interface of the other robot.

Python bridge between CoppeliaSim and ROS2

The bridge between CoppeliaSim and ROS2 was implemented in Python. Its purpose was to transfer the joint target values from the simulation to the command topics of the real robots. The complete bridge code is not printed in the main text; instead, the main text describes the logic of operation and shows only the necessary command and communication structure.

The bridge used two operating modes. The first mode was synchronization mode. In this mode, the real robot and the simulated robot were brought into the same or compatible joint configuration before execution. The second mode was run mode. In run mode, the bridge continuously published the joint target values from the simulation to the forward position controller command topic.

Pseudocode 5.1. Operating modes of the ROS2 communication bridge

```
INPUT:
    Measured joint states of the physical robot
    Joint targets generated by the simulation

WHILE the communication bridge is active DO
    IF synchronization mode is selected THEN
        Read the measured joint positions of the physical robot
        Arrange the values according to the simulated robot joint order
        Apply the measured positions to the simulated robot model

    ELSE IF run mode is selected THEN
        Read the latest joint targets generated by the simulation
        Arrange the values according to the controller joint order
        Publish the target positions to the corresponding ROS2 controller
    END IF
END WHILE
```

For the dual-robot case, the same principle was applied twice. The first set of simulated joint values was sent to the command topic of Robot 1, and the second set of simulated joint values was sent to the command topic of Robot 2. The topics were kept separated by namespaces, for example /robot1/forward_position_controller/commands and /robot2/forward_position_controller/commands.

Code listing 5.4. Forward position bridge startup commands

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

python3 ~/dual_coppelia_forward_position_bridge.py \
  --robot1-topic /robot1/coppelia_joint_target \
  --robot2-topic /robot2/coppelia_joint_target \
  --controller forward_position_controller
```

Extended bridge for recorded JointTrajectory execution

The initial bridge transferred only simulated joint positions to the forward position controller. In the extended experiments, a second Python workflow was added. It recorded the CoppeliaSim target positions, velocities and accelerations, converted them into complete JointTrajectory commands and sent them to the joint_trajectory_controller. The successful recorded command used time-scale 1.0, meaning that the recorded trajectory was replayed with the same timing as the recorded CoppeliaSim motion.

The recorded trajectory command used the following parameters: record-seconds defined the recording duration; time-scale = 1.0 kept the original recorded timing; downsample = 1 kept every recorded sample; filter-alpha = 0.30 smoothed the numerically calculated velocities and accelerations; max-start-error = 0.25 rad prevented execution if the robot was too far from the first trajectory point; max-velocity = 5.0 rad/s and max-acceleration = 5.0 rad/s^2 limited the values sent to the controller.

Synchronization before execution

Synchronization was required because an incompatible initial state between the physical and simulated robot caused a velocity-limit warning on the teach pendant. Therefore, before command transfer began, each simulated robot model was aligned with the measured joint state of the corresponding physical robot. An example of the warning observed after an unsynchronized start is shown in Figure 5.3.

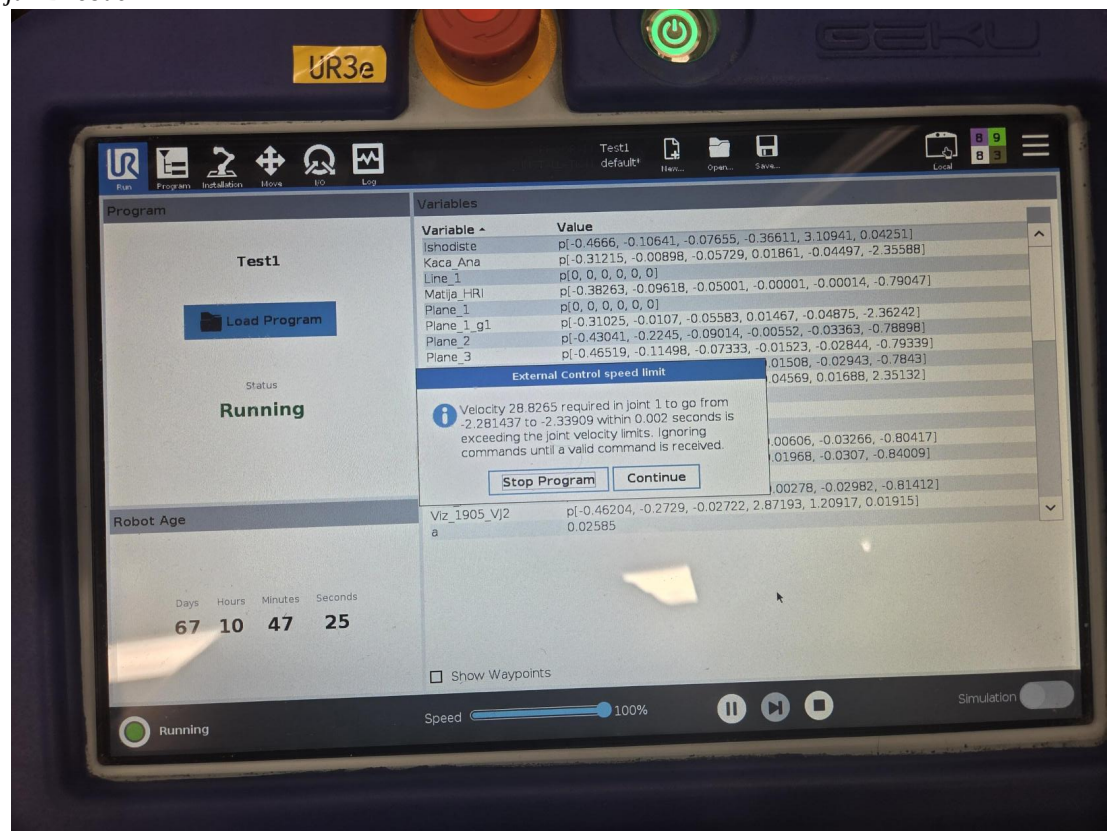


Figure 5.3. Teach pendant warning after unsynchronized start

5.4. Real-time transfer, validation and debugging

After synchronization, the system entered run mode. In this mode, the bridge continuously transferred joint targets from CoppeliaSim to ROS2. The forward position controller received the joint vector and forwarded it to the UR driver, which commanded the real robot. The same process was applied to both robots at the same time.

The confirmed result was that two real UR robots could follow two corresponding robots from the CoppeliaSim simulation. One real robot followed the motion of one simulated robot, while the second real robot followed the motion of the second simulated robot. This demonstrated that the connection was not only a single-robot proof of concept, but a dual-robot simulation-to-real connection.

The direct command approach also required that the commands sent from the simulation were achievable and smooth enough for the real robot. The forward position controller does not generate a complete high-level industrial trajectory by itself. It follows the target joint values provided by the user-side software. Therefore, the bridge and simulation must provide commands that are physically reasonable and consistent with the current robot state.

The CoppeliaSim scene used during the single-real-robot validation is shown in Figure 5.4.

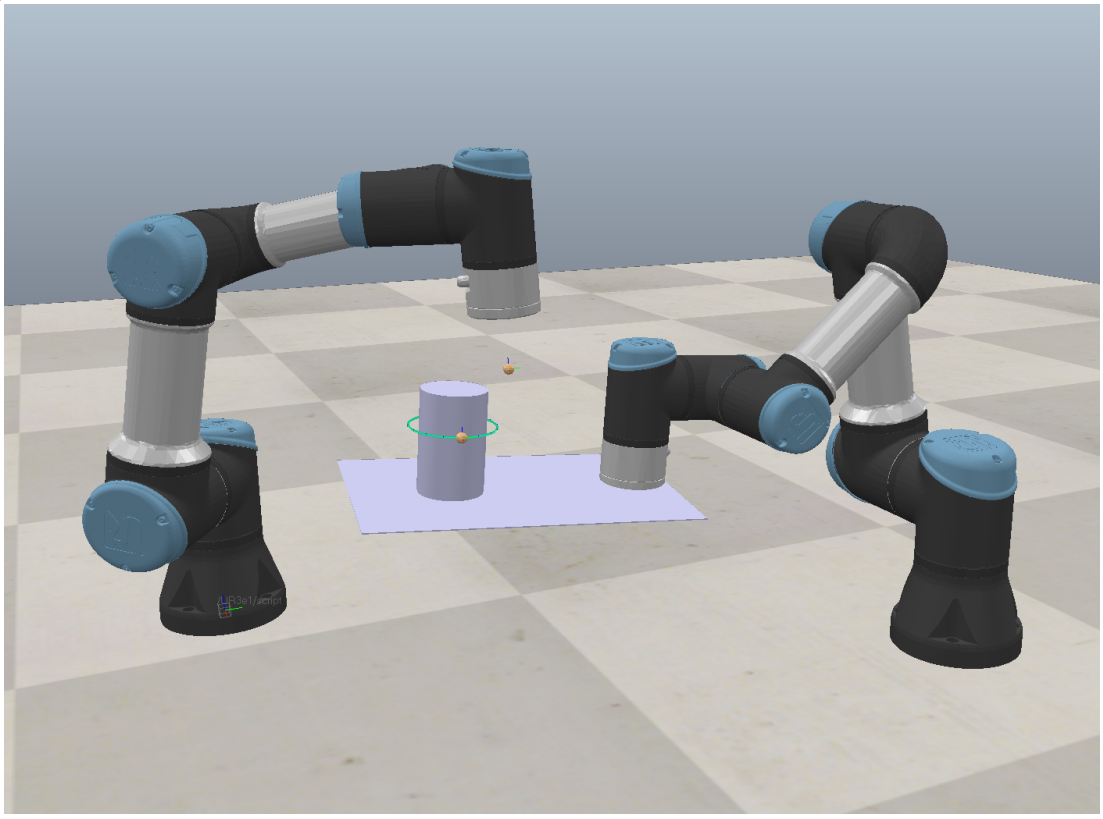


Figure 5.4. CoppeliaSim scene used for the single-real-robot validation

Single-robot validation and practical debugging

A practical issue appeared during repeated connection attempts. In some tests the UR driver connected successfully, while in other attempts the connection failed because the configuration package was not received within the default timeout. This behaviour was solved by increasing the RTDE initialization timeout in the UR client library used inside the workspace. After this modification, the driver connection became repeatable and both robots could be launched reliably. This step was important because the final system required two driver instances to start consistently every time.

Before the final dual-robot experiment, a separate single-robot test was carried out. In that step, only the UR3e was connected to CoppeliaSim through ROS2. The purpose was to determine whether a real robot could follow joint commands generated by the simulated robot at all. This was an important risk-reduction step: if one robot could not reliably follow the CoppeliaSim motion, there would be no justification for extending the system to two simultaneous robots.

The single-robot test confirmed that the CoppeliaSim-to-ROS2-to-UR command chain was functional. The UR3e followed the simulated motion sufficiently well, so the development

continued toward two real robots, separate namespaces, separate Python bridges and controller switching for both robot controllers.

Code listing 5.5. Essential Lua logic used in the single-real-robot validation

```
function publishCurrentJoints()
    local pos = {}
    for i = 1, #simJoints do
        pos[i] = sim.getJointPosition(simJoints[i])
    end

    simROS2.publish(pub, {
        name = rosJointNames,
        position = pos,
        velocity = {},
        effort = {}
    })
end

function sysCall_actuation()
    moveTargetAlongPath()
    simIK.handleGroup(ikEnv, ikGroup, {syncWorlds = true})
    publishCurrentJoints()
end
```

The teach pendant velocity warning shown during early tests confirmed why synchronization was necessary. If the simulated robot started from one joint configuration while the physical robot was in a significantly different configuration, the first streamed command required an unrealistically large change in a very short time. The controller therefore rejected the motion or displayed a velocity-limit warning. The final procedure avoids this problem by first synchronizing the simulated robot model to the measured real joint state and only then switching to run or track mode.

This connection phase represents one of the main practical results of the thesis. It showed that the developed simulation approach can be connected to real UR robots through ROS2 and that two robots can be operated simultaneously from corresponding simulated robot motions.

6. FINAL FUNCTIONAL SOLUTION

The final functional solution combines the developed CoppeliaSim simulation, the ROS2 communication layer and two real Universal Robots manipulators. The purpose of the final demonstration is to show that the system can execute a non-sequential task in which the motion of one robot changes the reference situation that the second robot must follow in real time. The final setup therefore goes beyond a simple animation or single-robot test: both real robots are connected, synchronized and commanded from the corresponding simulated robots.

6.1. Final demonstration scenario and object hierarchy

The final scene represents the last stage of the development, after the RoboDK phase, the simplified CoppeliaSim proof-of-concept and the single-real-robot CoppeliaSim-to-ROS2 validation. It consists of two Universal Robots manipulators connected to the corresponding simulated robots. In the final CoppeliaSim scene, the UR5e is the path-following robot, while the UR3e moves the path reference and executes the target-to-target motion. The path-following script therefore belongs to the UR5e side, and the motion-generating script belongs to the UR3e side.

The UR5e holds a pipe using a gripper and follows the moving circular reference. A circular path is defined in CoppeliaSim slightly in front of the UR3e arm. The moving target on this path is the reference used by the tracking robot. In the scene file this dummy is named target2, but in the explanation it is treated as the path-following target. The UR3e follows a sequence of spatial dummy targets named target3 to target10. These targets produce combined motion in several directions, including forward and backward motion, lateral motion, vertical displacement and orientation changes.

The task sequence is intentionally non-sequential, but the start is synchronized. First, the UR5e path-following robot slowly approaches the initial point of the circular path. After the UR5e reaches this point, it sends a ready signal to the UR3e script. Only then does the UR3e start moving through the target sequence target3 to target10. At the beginning of the UR3e motion, the UR5e keeps the reached relative position to confirm that it can adapt to the changing UR3e pose. After the defined delay, the UR5e starts following the circular path while the UR3e continues its own target-to-target motion.

Simulation scene and object hierarchy

The final CoppeliaSim scene contains the two robot models, the circular path with its moving target, the target sequence target3 to target10 and the scripts used for synchronization, tracking and path execution. The scene hierarchy is important because the path is attached to the motion-generating robot structure, while the path-following robot reads the moving target and generates its own joint commands. In the corrected final scene hierarchy, UR5e1 is the tracking robot and UR3e2 is the robot that moves the path reference. The final scene also includes a gripper and pipe model so that the collision-critical part of the task is visually represented.

The complete final CoppeliaSim dual-robot scene is shown in Figure 6.1.

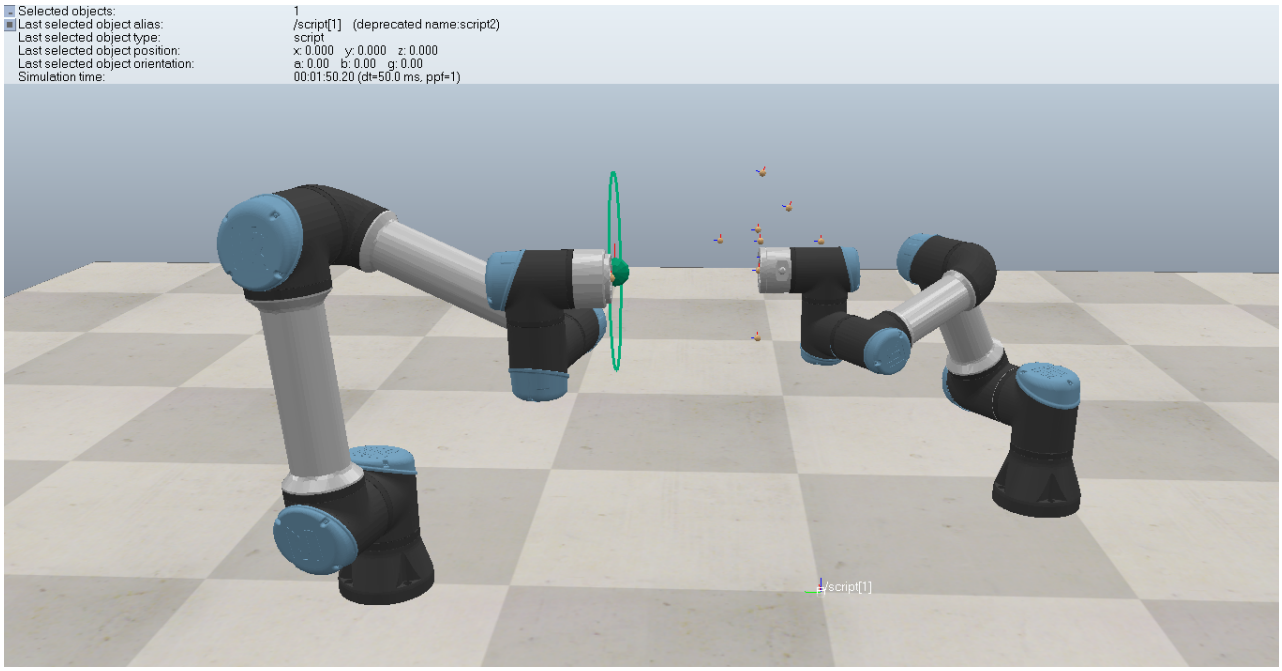


Figure 6.1. Final CoppeliaSim dual-robot scene

The annotated motion views in Figures 6.2 to 6.4 clarify the robot motion executed during the final task. The red arrows mark the target-to-target reference motion of the UR3e. The UR5e first follows the changing UR3e reference from the reached path position, and after the defined delay it starts moving along the green circular path shown in the images.

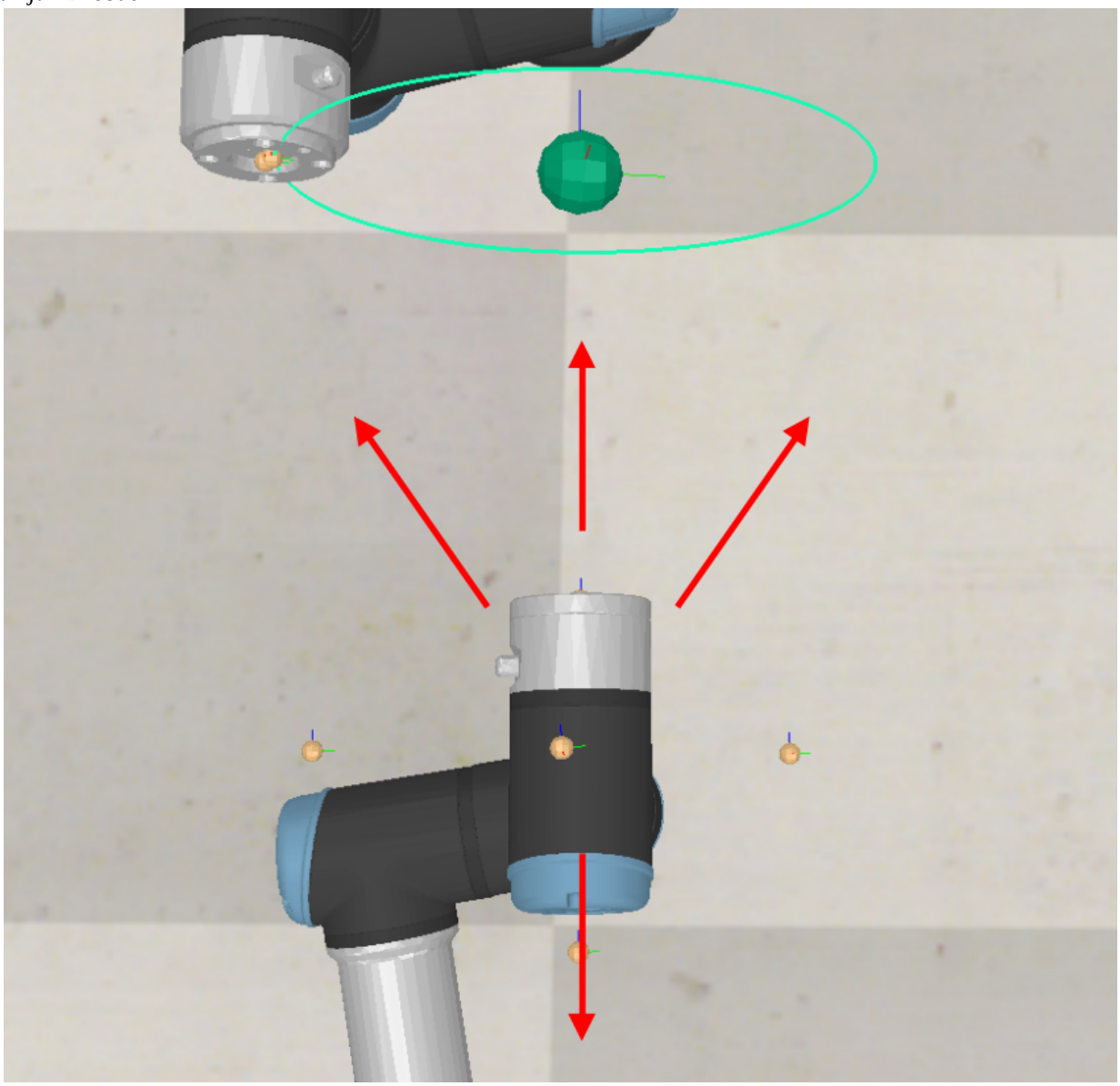


Figure 6.2. Annotated top view of the UR3e reference motion and the UR5e circular path.

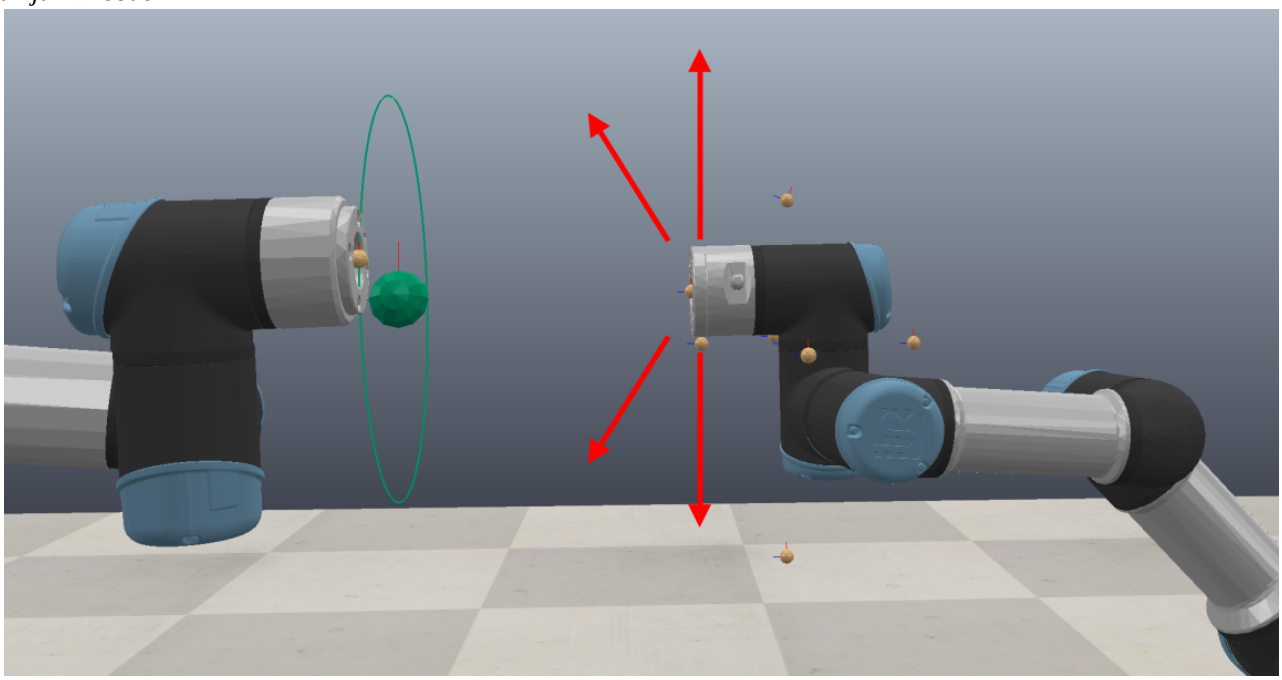


Figure 6.3. Annotated side view of the UR3e reference motion and the UR5e circular path.

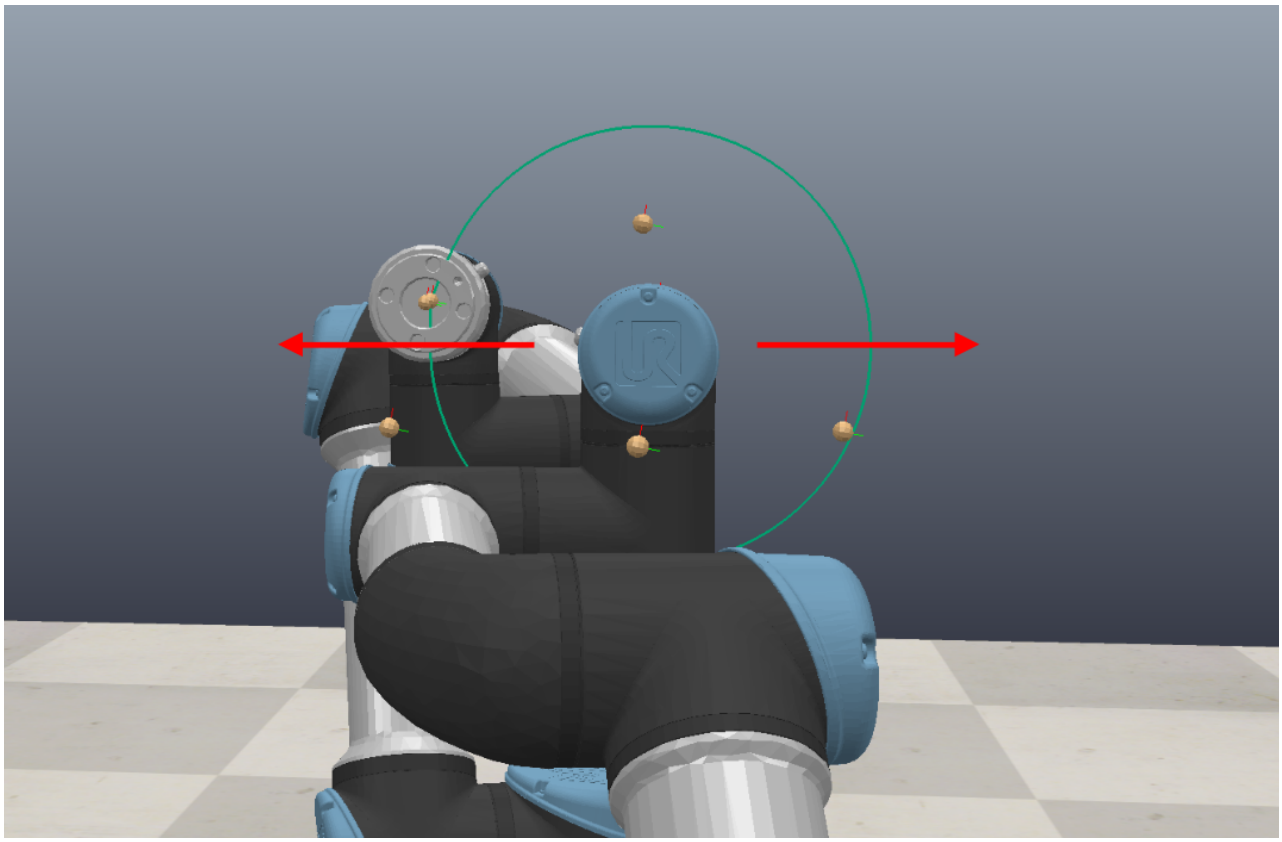


Figure 6.4. Annotated back view of the UR3e reference motion and the circular path.

The path script moves the target dummy along the circular path after a selected start time. This delay is useful because the tracking robot first needs to approach the initial point of the path slowly. After the start time is reached, the target moves along the path with a defined velocity and acceleration. The final scripts are separated according to robot function: script[0]

belongs to the robot that follows the path, while script[1] belongs to the robot that moves the path reference and executes the target-to-target motion.

Code listing 6.1. UR5e forward position path-following logic

```
function sysCall_actuation()
    local dt = sim.getSimulationTimeStep()

    if MODE == 'sync' then
        applyMeasuredJointStateToSimulation()
        return
    end

    if MODE == 'track' then
        local pGoal = getCurrentPointOnCircularPath()
        moveProxyTargetToward(pGoal, maxLinSpeed, dt)
        simIK.handleGroup(ikEnv, ikGroup, {syncWorlds = true})
        publishCurrentJointTarget('/robot2/coppelia_joint_target')
    end
end
```

Code listing 6.2. UR3e forward position motion-generating logic

```
function sysCall_actuation()
    if MODE == 'sync' then
        applyMeasuredJointStateToSimulation()
        return
    end

    if MODE == 'run' then
        local ready = sim.getInt32Signal(WAIT_SIGNAL)
        if ready ~= 1 then
            publishCurrentJointTarget('/robot1/coppelia_joint_target')
            return
        end

        moveProxyTargetToward(targets[currentTargetIndex], maxLinSpeed, dt)
        simIK.handleGroup(ikEnv, ikGroup, {syncWorlds = true})
        publishCurrentJointTarget('/robot1/coppelia_joint_target')
    end
end
```

6.2. Synchronization and ROS2 start-up procedure

Before execution, the real robots are placed in arbitrary starting positions. The first step of the procedure is synchronization: the simulated models read the joint states of the physical robots and adjust their simulated joint positions to match the real configurations. This step is necessary because the bridge later sends joint positions from the simulation to the real robot. If synchronization is skipped, the real robot may receive a command that is too far from its current physical position, resulting in a velocity-limit warning or command rejection on the teach pendant.

The complete synchronization and execution workflow used in the final experiment is shown in Figure 6.5.

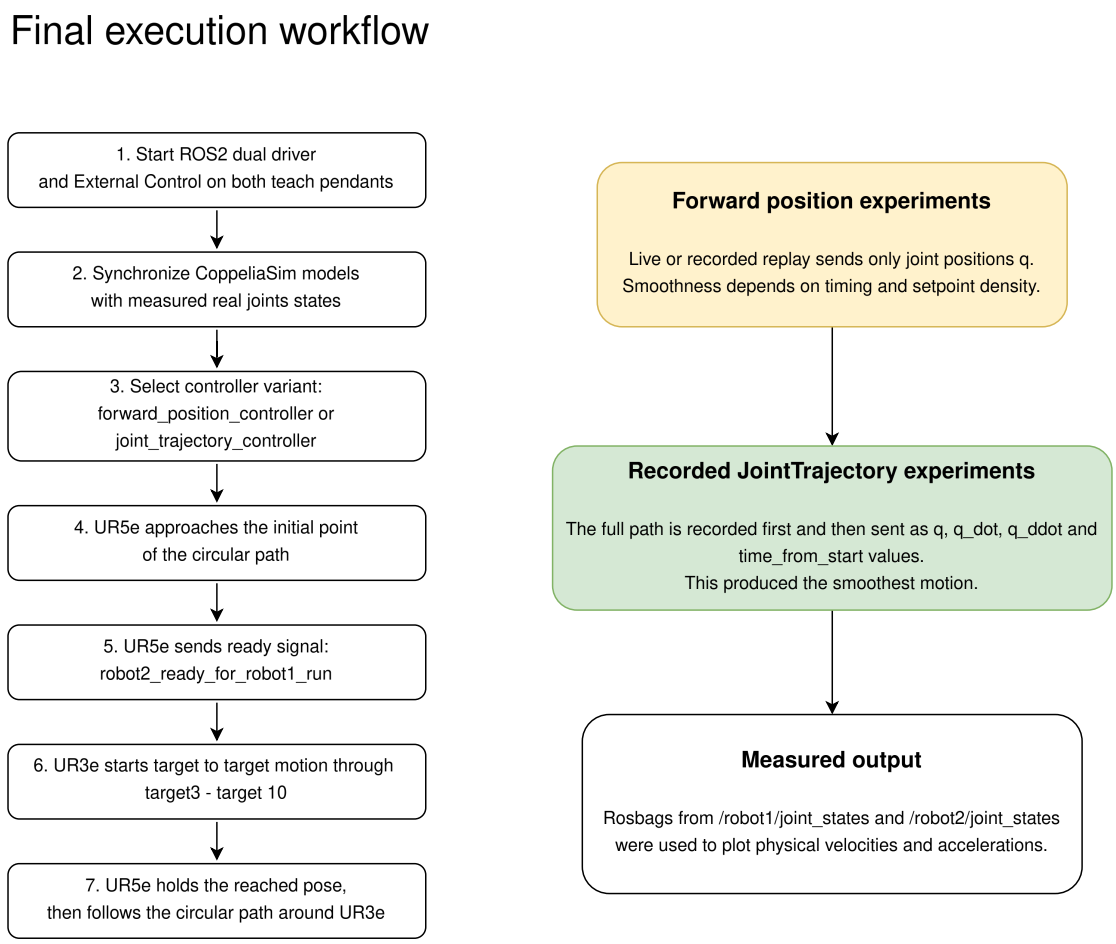


Figure 6.5. Experimental workflow and final execution sequence

The ROS2 part of the final setup uses a common ROS domain and separate namespaces for both robots. The launch file starts both robot drivers, while the teach pendants run the External Control program. Two execution variants were then evaluated. In the first variant, separate Python bridge scripts transferred CoppeliaSim joint positions to the forward_position_controller. In the second variant, Python recorded the simulated positions, velocities and accelerations and sent complete JointTrajectory commands to the joint_trajectory_controller.

Code listing 6.3. Forward position bridge command used for live and recorded replay tests

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

python3 ~/dual_coppelia_forward_position_bridge.py \
  --mode recorded-or-live \
  --controller forward_position_controller
```

Code listing 6.4. Recorded JointTrajectory execution command

```
export ROS_DOMAIN_ID=7
source /opt/ros/humble/setup.bash
source ~/ur_ws/install/setup.bash

python3 ~/dual_coppelia_record_then_joint_trajectory_wait_key.py \
  --record-seconds 120.0 \
  --time-scale 1.0 \
  --downsample 1 \
  --filter-alpha 0.30 \
  --max-start-error 0.25 \
  --max-velocity 5.0 \
  --max-acceleration 5.0
```

The parameters in the recorded JointTrajectory command have the following roles. record-seconds defines the length of the CoppeliaSim recording. time-scale was set to 1.0, so the recorded trajectory is executed with the same time scale as the recorded simulation motion. downsample = 1 means that every recorded sample is used. filter-alpha defines the amount of filtering applied to the calculated velocity and acceleration signals. max-start-error prevents execution if the real robot is too far from the first recorded trajectory point. max-velocity and max-acceleration limit the velocity and acceleration values sent in the trajectory.

6.3. Experimental validation and final result

The final system was developed through three validation stages. The first stage was the CoppeliaSim simulation proof-of-concept, where available robot models were used to confirm that a robot could follow a moving path created by the motion of another robot. The second stage was the single-real-robot validation, where the motion of the simulated path-following UR robot was transferred through ROS2 to one real UR3e robot. The third stage was the final dual-real-robot experiment, where both robots were connected, synchronized and commanded from the simulation at the same time.

In the final stage, both robots were launched in separate namespaces and controlled through separate bridge scripts and controller managers. The final demonstration combined the UR3e target-to-target motion with the UR5e path-following task.

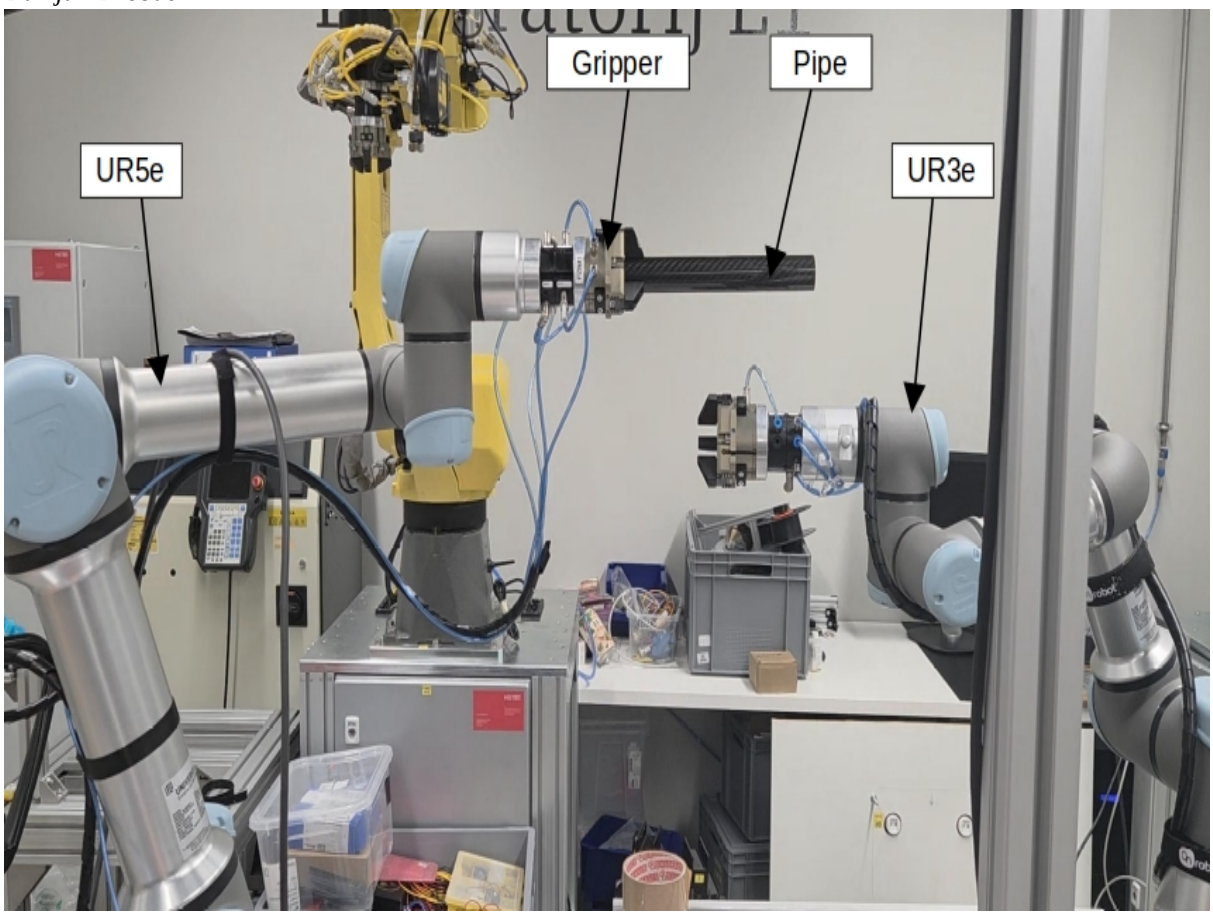


Figure 6.6. Physical dual-robot laboratory setup

The physical laboratory setup used for the final dual-robot validation is shown in Figure 6.6. The annotated labels identify the main elements of the experiment: the UR3e robot, the UR5e robot, the gripper and the pipe. In the final task, the UR3e acts as the motion-generating robot and executes the target-to-target reference motion defined in the CoppeliaSim scene. The UR5e acts as the path-following robot. It carries the pipe with the gripper and follows the circular reference path around the UR3e arm. This setup was used to validate whether the coordinated CoppeliaSim-defined motion could be transferred to two real UR robots through ROS2.

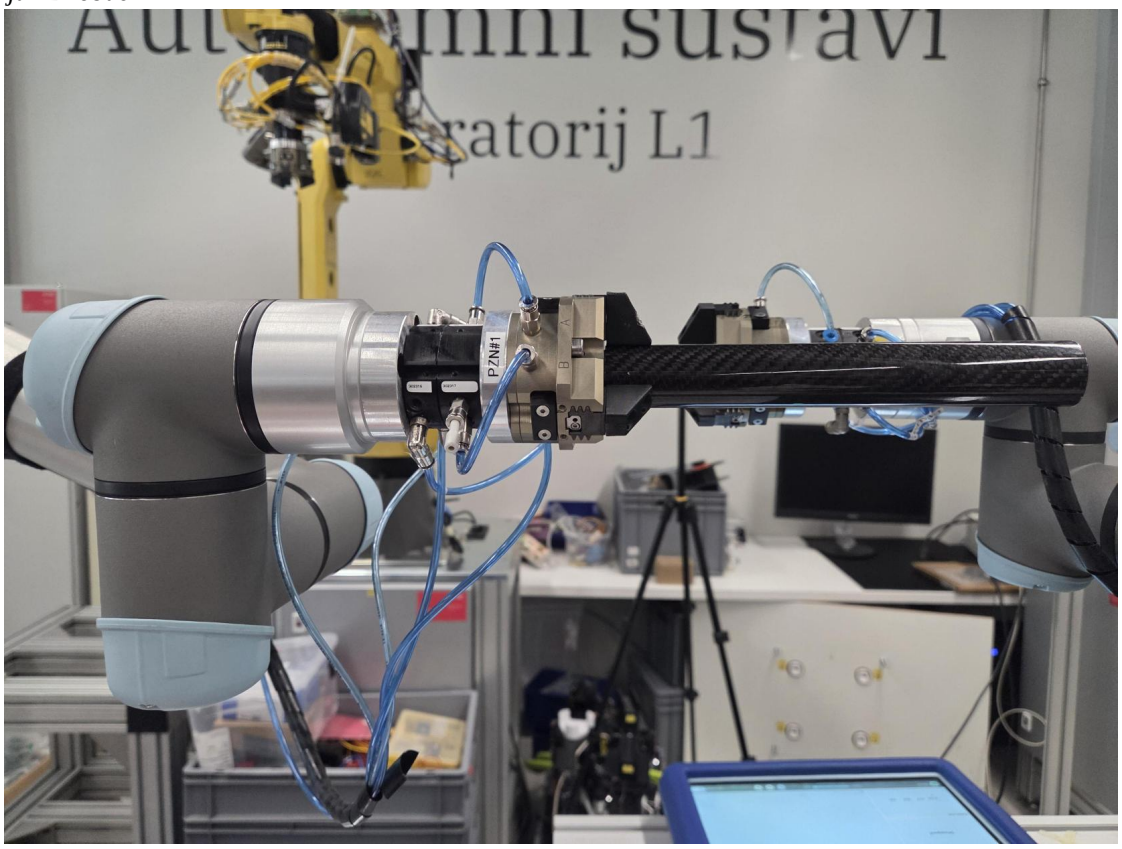


Figure 6.7. Close-up of the gripper and pipe used in the final demonstration

A close-up of the gripper and pipe used during the final experiment is shown in Figure 6.7. This view illustrates the physical element whose clearance was checked while the path-following robot moved the pipe around the other robot arm.

Result of the final demonstration

The final demonstration confirmed that two physical UR robots could execute the developed non-sequential task simultaneously while following the corresponding motions generated in CoppeliaSim. The UR3e executed the target-to-target motion and changed the reference situation, while the UR5e followed the moving circular path and carried the pipe around the UR3e arm. The physical execution therefore reproduced the main task defined in the final simulation scene, with both robots operating at the same time rather than as two independent sequential programs.

To evaluate the final demonstration quantitatively, the CoppeliaSim joint-position references and the measured joint positions of both physical robots were recorded using rosbag2 [11] during the final live forward-position execution. The experiment was performed with the forward position controller. A five-second interval was selected to provide a detailed comparison of the simulated and physical robot motion.

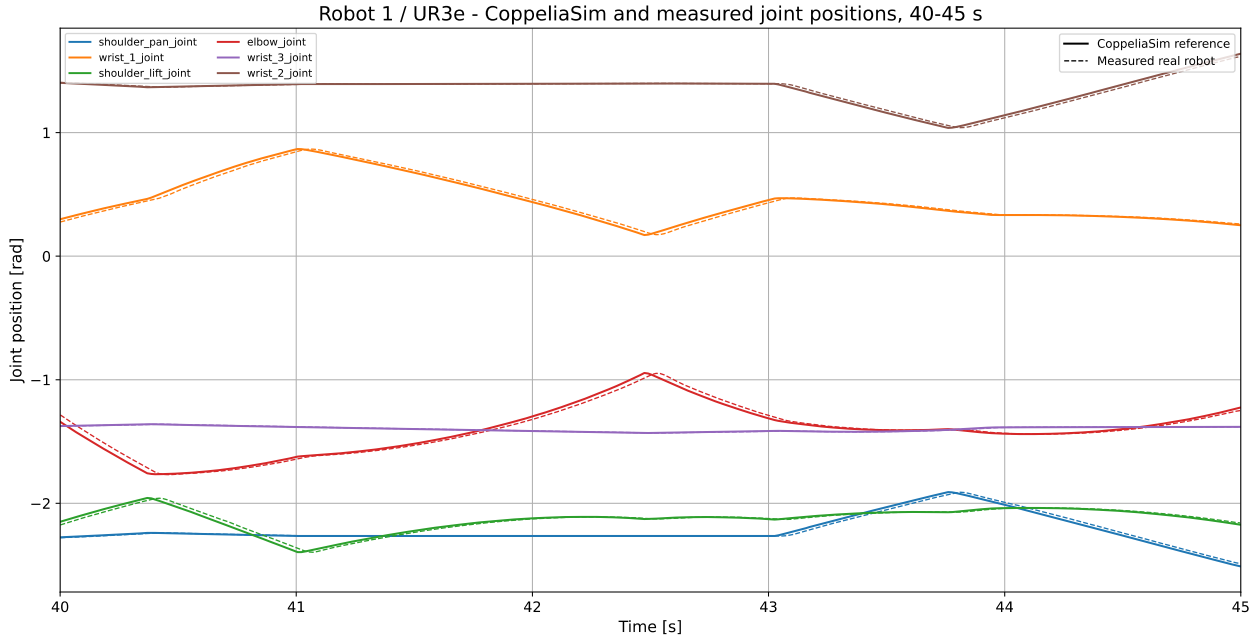


Figure 6.8. Comparison of CoppeliaSim reference and measured UR3e joint positions

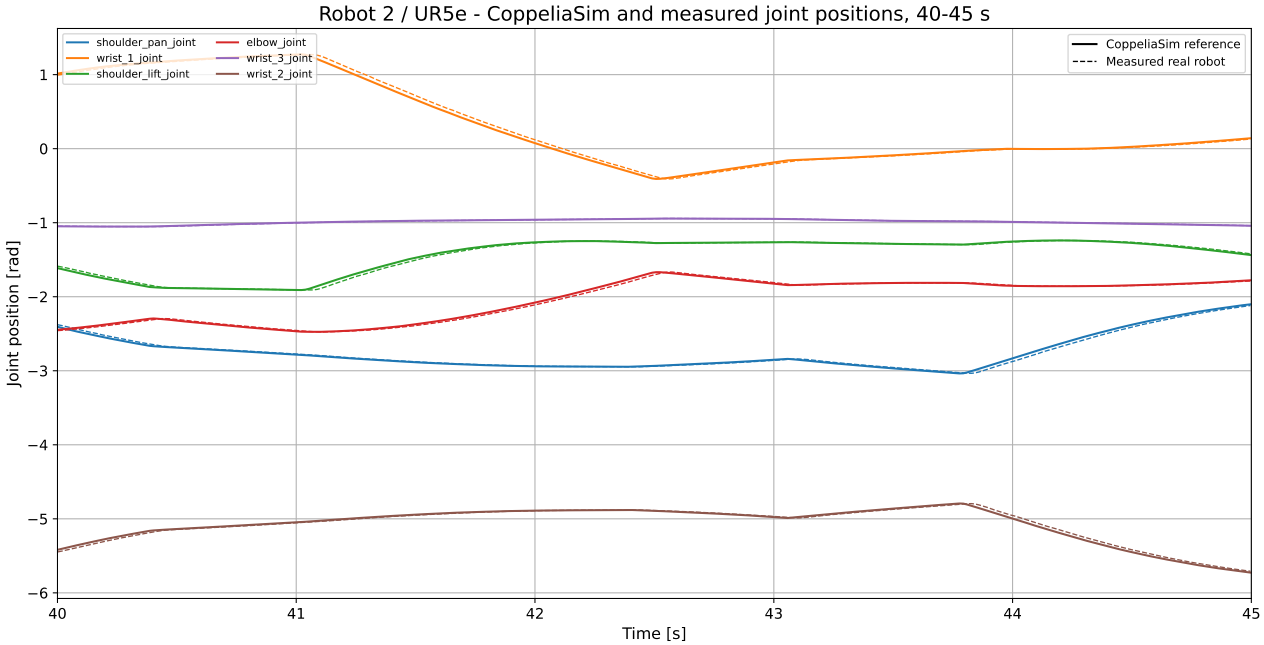


Figure 6.9. Comparison of CoppeliaSim reference and measured UR5e joint positions

Figures 6.8 and 6.9 compare the reference joint positions generated in CoppeliaSim with the measured joint positions of the physical UR3e and UR5e. Solid lines represent the CoppeliaSim references, while dashed lines represent the measured physical robot positions. For both robots, the measured trajectories follow the shape and timing of the corresponding

simulated references over all six joints. The curves largely overlap during the selected interval, while the most visible differences occur during faster changes of joint position. These results confirm that the joint-space motion generated in CoppeliaSim was transferred to both physical robots during simultaneous execution.

The joint-position comparison confirms the simulation-to-real transfer for each robot separately. However, the final task also requires the relative spatial motion of both robots to be evaluated in the same coordinate system. Therefore, the measured TCP trajectories were transformed into the common CoppeliaSim world/laboratory reference frame using the recorded TF data and the defined poses of both robot bases.

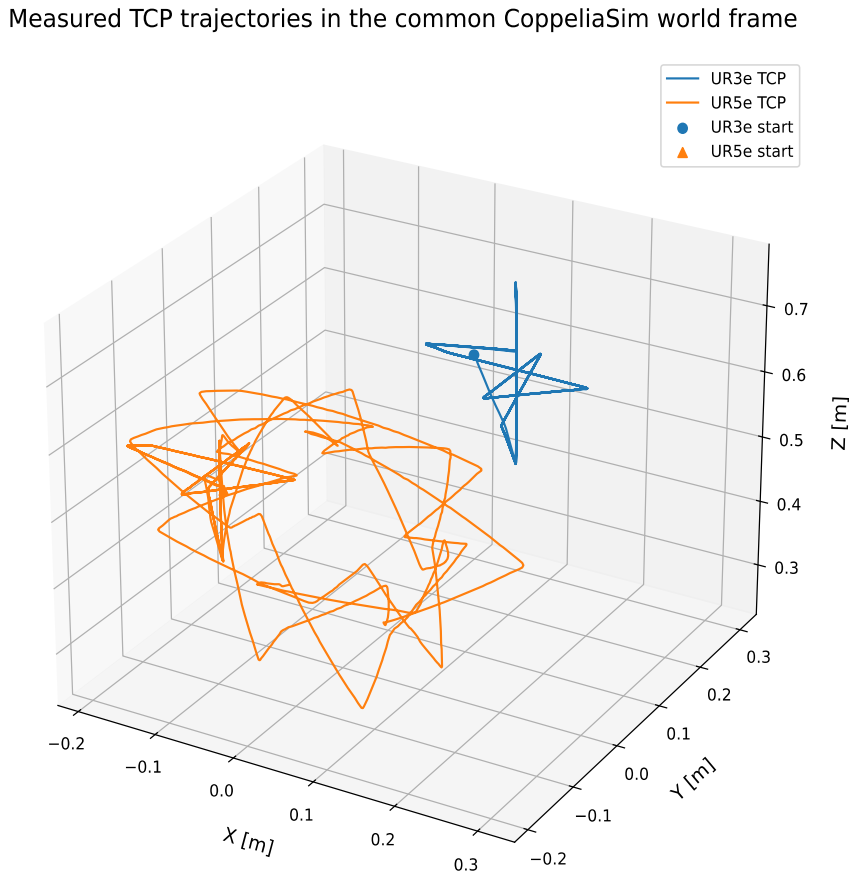


Figure 6.10. Measured TCP trajectories of the UR3e and UR5e in the common CoppeliaSim world/laboratory coordinate system

Figure 6.10 shows the measured TCP trajectories of both robots expressed in the same Cartesian coordinate system. Unlike two independent robot plots, this representation directly shows the relative spatial placement of the UR3e and UR5e trajectories and the workspace regions occupied by their TCPs during the recorded task. The shared-coordinate

representation therefore makes the relative position and motion of the two robot TCPs directly visible.

Together, the joint-position comparisons and the common-coordinate TCP trajectory plot provide a quantitative validation of the final demonstration. The first two graphs show that both physical robots followed the corresponding CoppeliaSim joint references, while the common XYZ graph shows their relative Cartesian motion during the coordinated dual-robot task.

Additional controller validation

After the first functional dual-robot execution with the forward position controller, the pipe did not visibly vibrate, but slight vibration could be felt on the robot arms when touched. This observation motivated additional controller experiments aimed at improving the smoothness of the real-robot motion.

For this reason, the same CoppeliaSim task was executed with live and recorded variants of the forward position controller and with recorded variants of the joint trajectory controller. These experiments are analyzed in Chapter 7 using joint-state measurements from the real robots.

7. ANALYSIS AND COMPARISON OF APPROACHES

This chapter extends the earlier comparison of simulation tools with a controller-level analysis. The first functional CoppeliaSim-to-real-robot solution used the forward position controller, which was sufficient to prove that the two real robots could follow the simulated task. The additional experiments compare how command timing and controller input information influence the smoothness of the real robot motion.

Six experiments were selected for the final comparison. They use the same non-sequential CoppeliaSim task, but differ in controller type, live or recorded execution, and the CoppeliaSim real-time toggle setting. The velocity curves were obtained from the real robot joint_states topics, while acceleration was calculated from consecutive measured velocity samples. Therefore, the plots represent the physical robot response rather than only the simulated target motion.

7.1. Controller command representations

The two main controller approaches differ in the information that is sent to the robot. The forward position controller receives only a vector of joint positions. The joint trajectory controller can receive a complete time-defined trajectory in which every point contains joint positions, velocities, accelerations and a time_from_start value. This distinction was the main reason for the controller comparison.

7.2. Real-time toggle interpretation

The CoppeliaSim real-time toggle changes the relationship between simulation time and physical wall time. With the toggle enabled, the parameters used in the Lua scripts correspond to real physical units, so the simulated motion duration matches real time. With the toggle disabled, the simulator is not limited by wall time. In this setup the same simulated motion was generated approximately three times faster in real time, so the total physical duration was about three times shorter and the effective physical speeds were higher.

The real-time toggle does not change the geometric path or the control logic. It changes how quickly the simulation generates the target samples in physical time. Therefore, the same robot path can appear faster or slower in the real laboratory depending on whether the toggle is enabled.

7.3. Computation of velocities and accelerations in CoppeliaSim

For the JointTrajectory experiments, the Lua scripts were extended so that the target topic contained not only joint positions but also joint velocities and accelerations. The current joint positions were read from the simulated robot. The joint velocity was calculated as the difference between the current and previous joint position divided by the simulation time step. The joint acceleration was calculated as the difference between the current and previous joint velocity divided by the same time step.

Because the robot joints are rotational, the position difference was calculated using wrapToPi. This prevents a transition through the -pi / +pi boundary from being interpreted as a large artificial jump. The raw velocity and acceleration values were then filtered with a first-order filter and limited to selected maximum values before publishing.

The CoppeliaSim target topic used the JointState message format. The field position contained q, the field velocity contained q_dot, and the field effort was used to transfer q_ddot because JointState does not contain a dedicated acceleration field.

Code listing 7.1. Computation and transfer of joint velocities and accelerations

```
for i = 1, #simJoints do
    local dq = wrapToPi(currentPos[i] - lastPos[i])
    rawVel[i] = dq / dt
    rawAcc[i] = (rawVel[i] - lastVel[i]) / dt

    vel[i] = velAlpha * rawVel[i] + (1.0 - velAlpha) * filteredVel[i]
    acc[i] = accAlpha * rawAcc[i] + (1.0 - accAlpha) * filteredAcc[i]
end

simROS2.publish(pubState, {
    name = rosJointNames,
    position = currentPos,
    velocity = vel,
    effort = acc
})
```

7.4. Experimental matrix

The six controller experiments and their main observations are listed in Table 7.1.

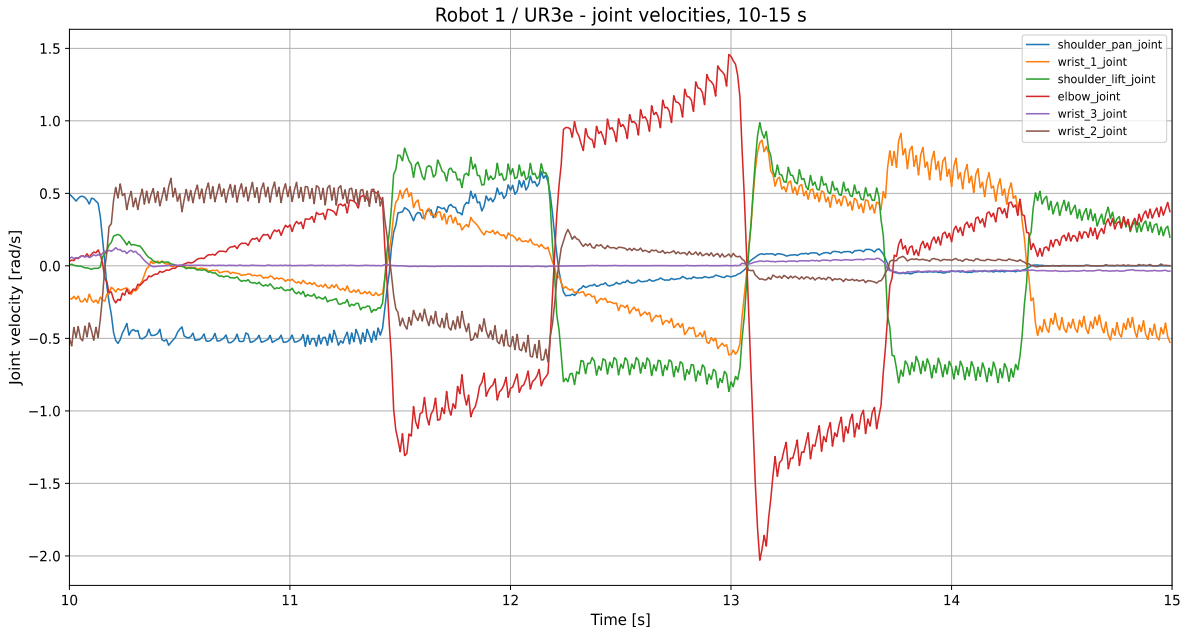
Table 7.1. Controller experiments used for the final comparison

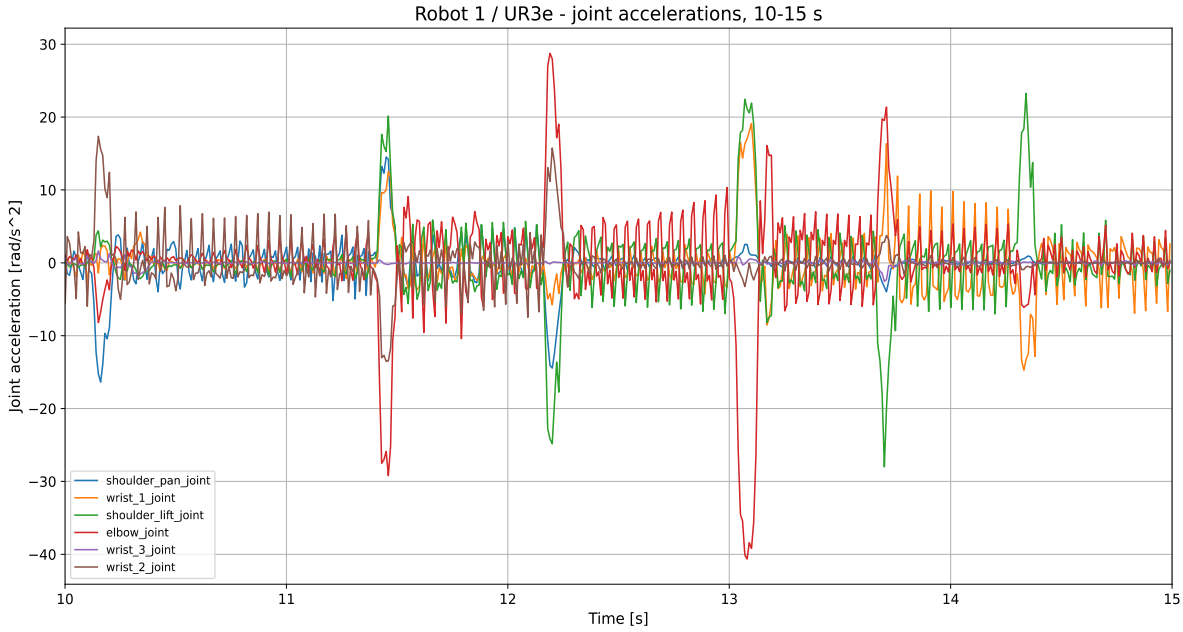
Experiment	Method	RT	Observation
FP live no-RT	FPC live	OFF	Good tracking; no visible pipe vibration; slight vibration felt on the robot arm when touched.
FP live RT	FPC live	ON	Slower motion, but more visible pipe vibration.
FP replay no-RT	FPC recorded replay	OFF	Similar to live no-RT; replay did not significantly change the result.
FP replay RT	FPC recorded replay	ON	Better than live RT; visible pipe vibration was reduced.
JT recorded no-RT	JTC recorded trajectory	OFF	No visible vibration and no vibration felt on the robot arm.
JT recorded RT	JTC recorded trajectory	ON	No vibration; stable motion with real-time duration.

7.5. Forward position live experiments

In the live forward position experiments, the CoppeliaSim Lua scripts continuously published joint positions, and the Python bridge immediately forwarded these positions to the forward position controller. The controller received only the current joint position vector. It did not receive the future trajectory, joint velocities or joint accelerations.

In the no-real-time case, the simulator generated target samples faster in physical time. This produced a dense stream of position commands, so the pipe did not visibly vibrate even though the real motion was faster. In the real-time case, the command stream matched physical time and the same motion took longer. The position-only reference became more discrete in wall time, and the pipe vibrated more visibly.


Robot 1 / UR3e - joint velocities, 10-15 s
Joint velocity [rad/s]
Time [s]
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint


Robot 1 / UR3e - joint accelerations, 10-15 s
Joint acceleration [rad/s^2]
Time [s]
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint

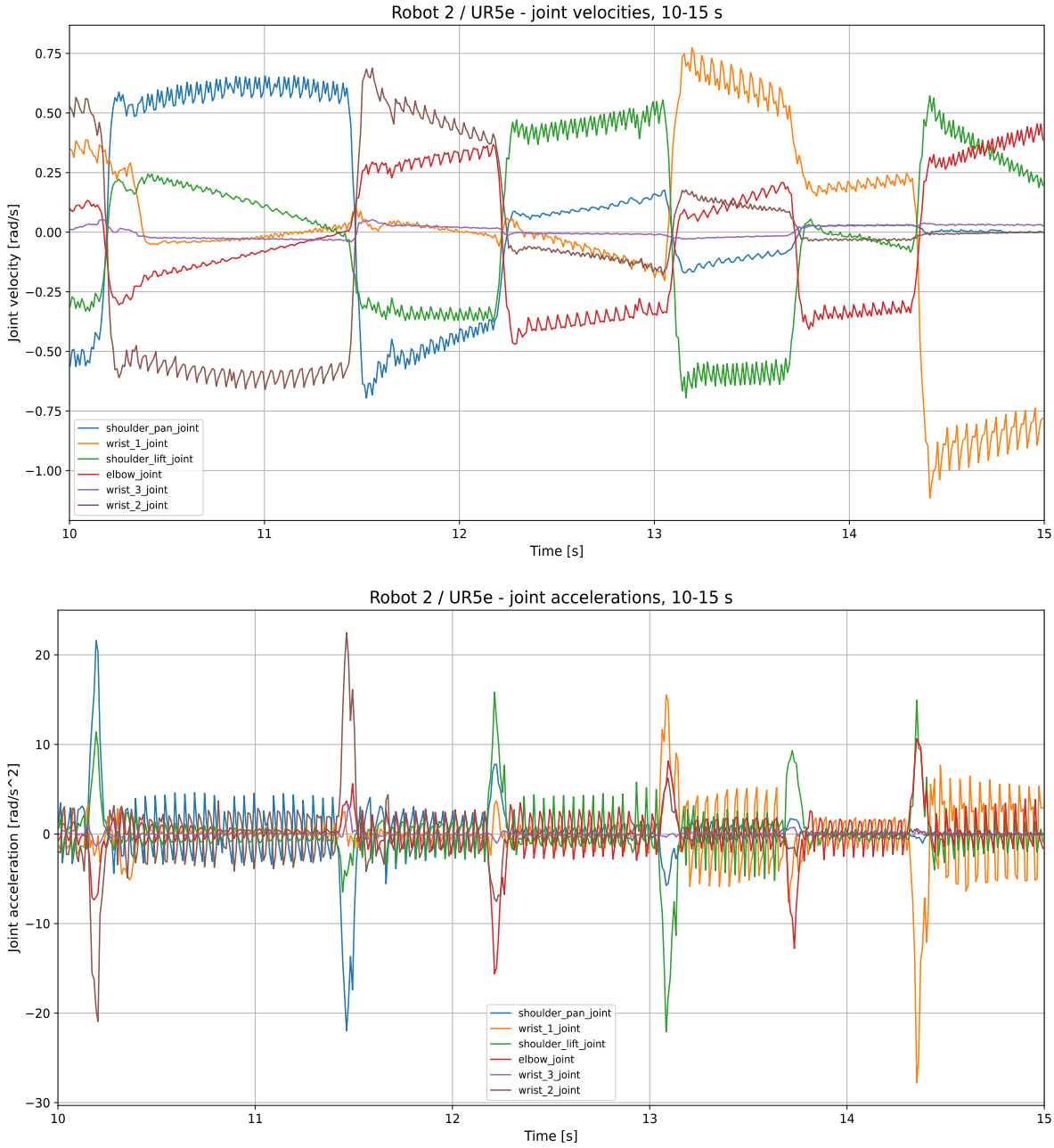
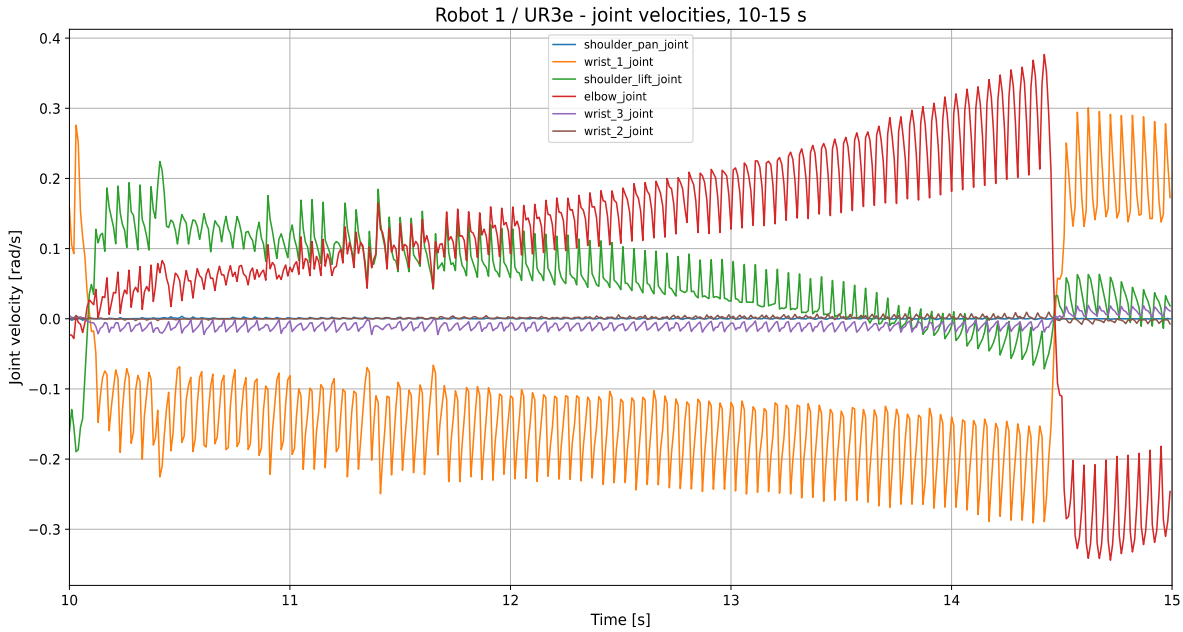
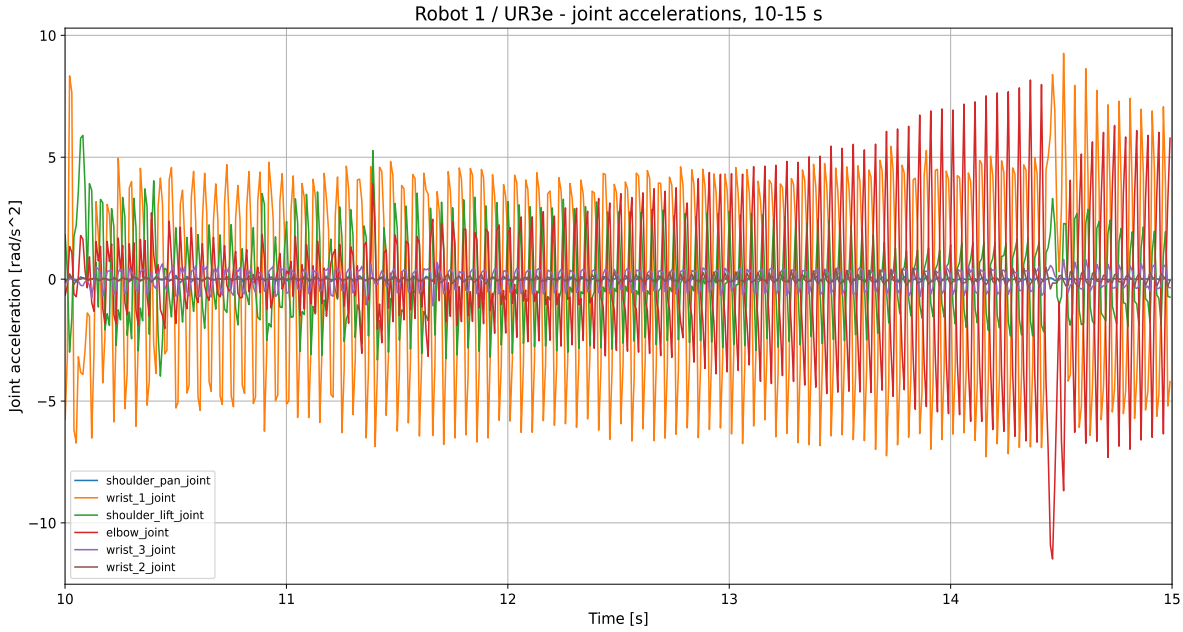


Figure 7.1. Forward position live execution without real-time toggle: measured joint velocities and accelerations

Figure 7.1 shows that the no-real-time live experiment produced higher physical velocities and acceleration peaks because the same simulated motion was executed in a shorter physical time. However, the command stream was dense enough that the pipe did not visibly vibrate.


Robot 1 / UR3e - joint velocities, 10-15 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint velocity [rad/s]
Time [s]


Robot 1 / UR3e - joint accelerations, 10-15 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint acceleration [rad/s^2]
Time [s]

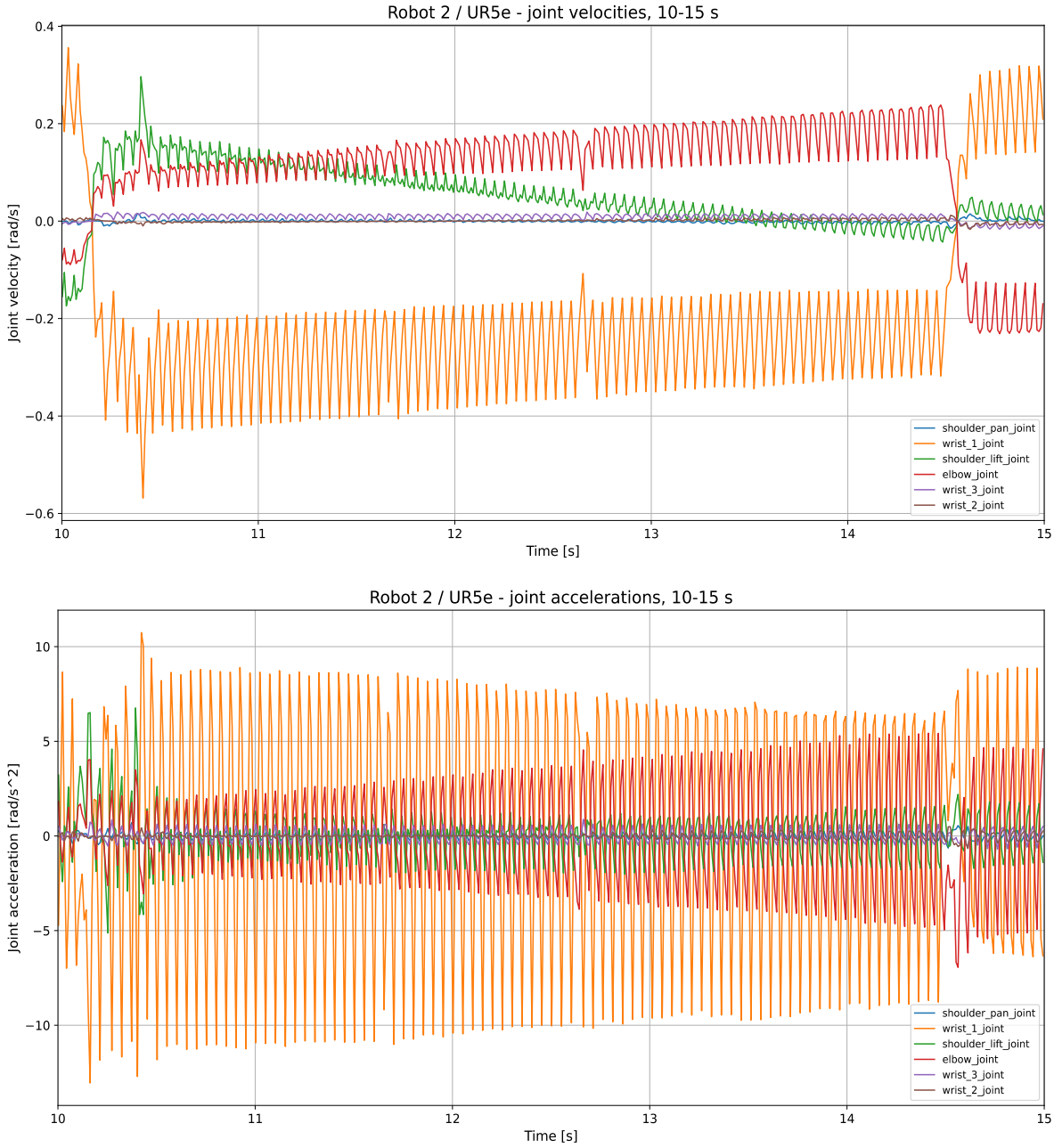


Figure 7.2. Forward position live execution with real-time toggle: measured joint velocities and accelerations

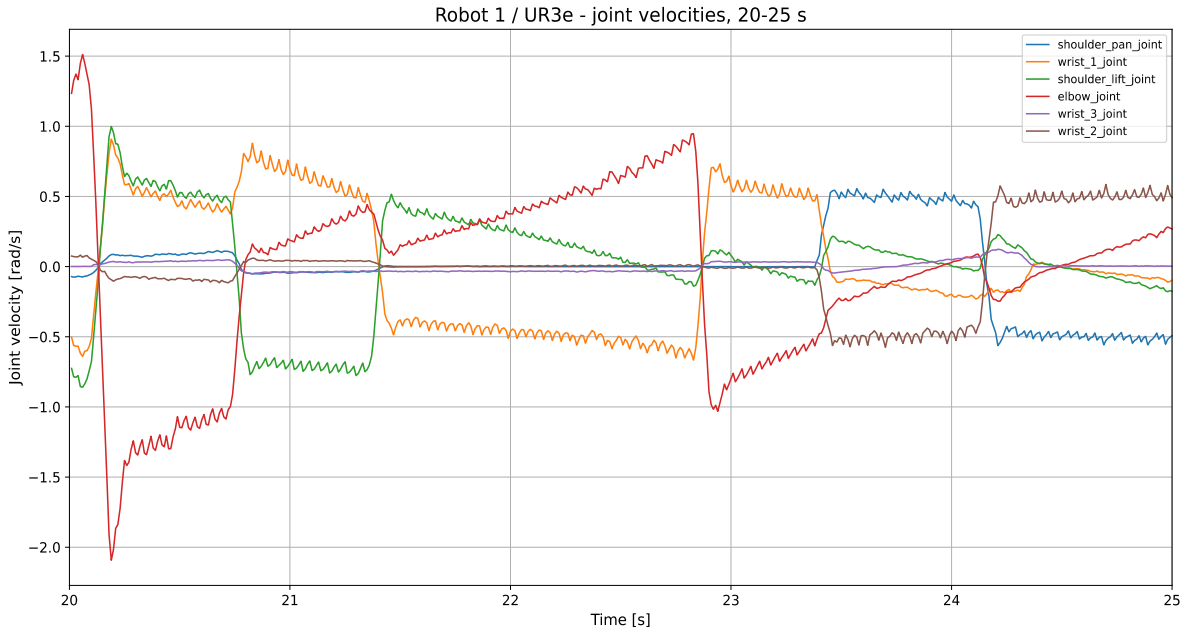
Figure 7.2 shows lower mean joint velocities than the no-real-time case, but frequent step-like velocity changes and alternating acceleration peaks. The forward position controller received only discrete joint-position setpoints; no target velocities, accelerations, future trajectory points or time_from_start values were provided. Each position reference was therefore effectively held until the next message arrived. With the CoppeliaSim real-time mode enabled, the command stream was less dense in wall time, which made the change between successive position references more pronounced. The robot reacted to each new reference with a short corrective motion, producing the observed fluctuations in the measured joint

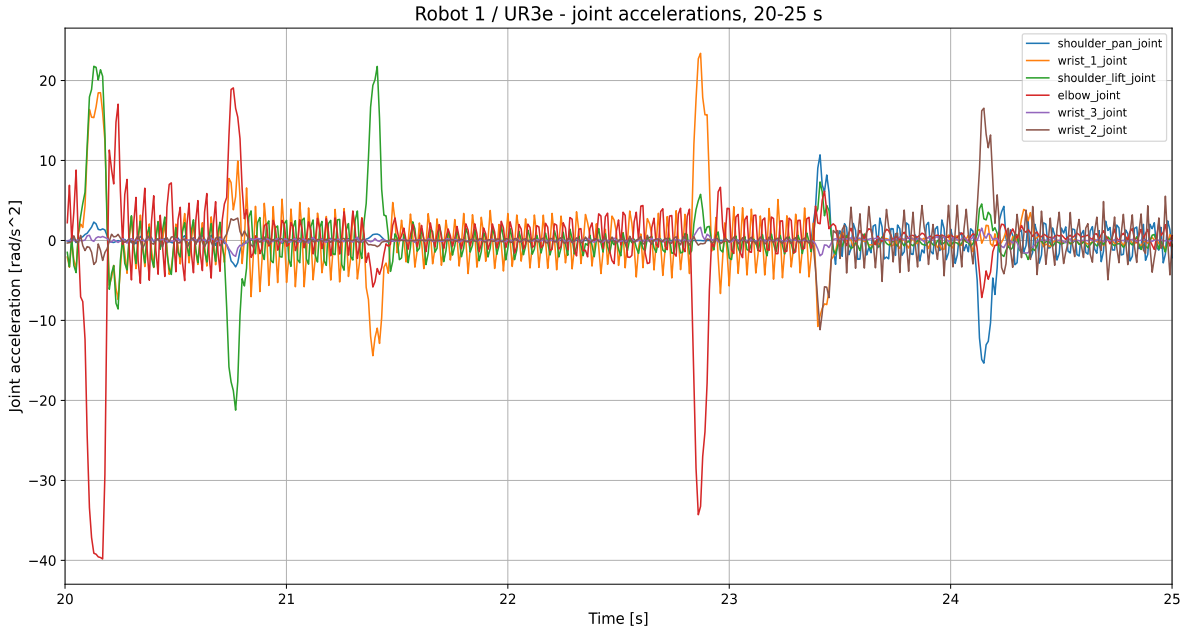
velocity. Since acceleration was calculated from consecutive joint-state velocity samples, numerical differentiation additionally amplified small velocity variations, timing irregularities and measurement noise into sharp alternating peaks. The oscillatory curves therefore resulted from both the discrete position-only command stream and derivative amplification, rather than from a deliberately commanded acceleration profile. This behaviour was consistent with the observed pipe vibration and motivated the subsequent fixed-rate replay and complete time-defined JointTrajectory experiments.

7.6. Forward position recorded replay experiments

In the recorded forward position experiments, CoppeliaSim was used only to generate and record the joint positions. During physical execution, Python replayed the recorded positions to the forward position controller at a regular publish rate. The controller still received only joint positions, but the live Coppelia timing was removed from the physical execution.

The recorded no-real-time experiment behaved almost the same as the live no-real-time experiment because the live no-real-time stream was already dense and regular enough. The recorded real-time experiment, however, was much better than the live real-time experiment because Python replay sent the positions with a more regular timing than the direct live Coppelia stream.


Robot 1 / UR3e - joint velocities, 20-25 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint velocity [rad/s]
Time [s]


Robot 1 / UR3e - joint accelerations, 20-25 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint acceleration [rad/s^2]
Time [s]

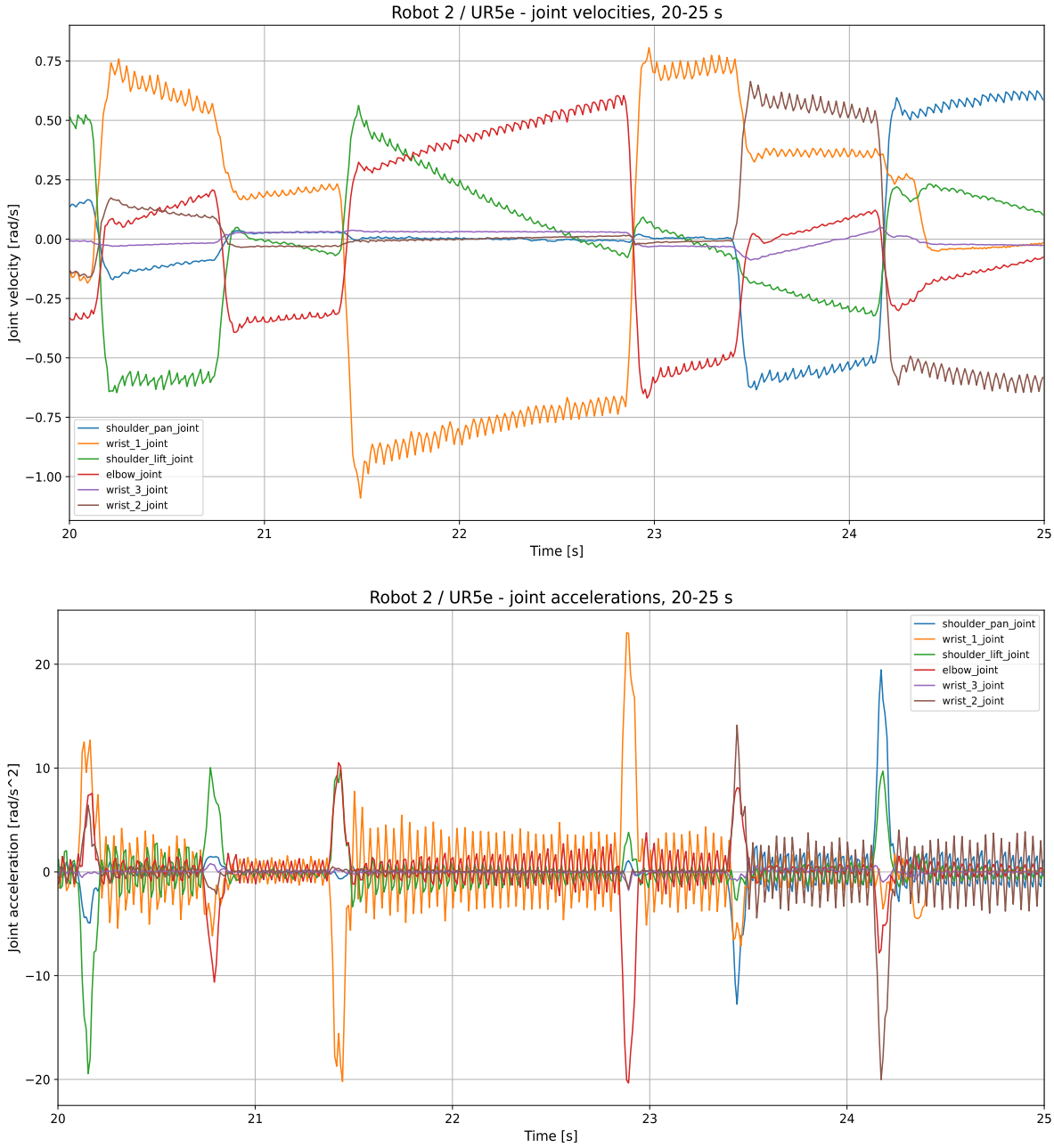
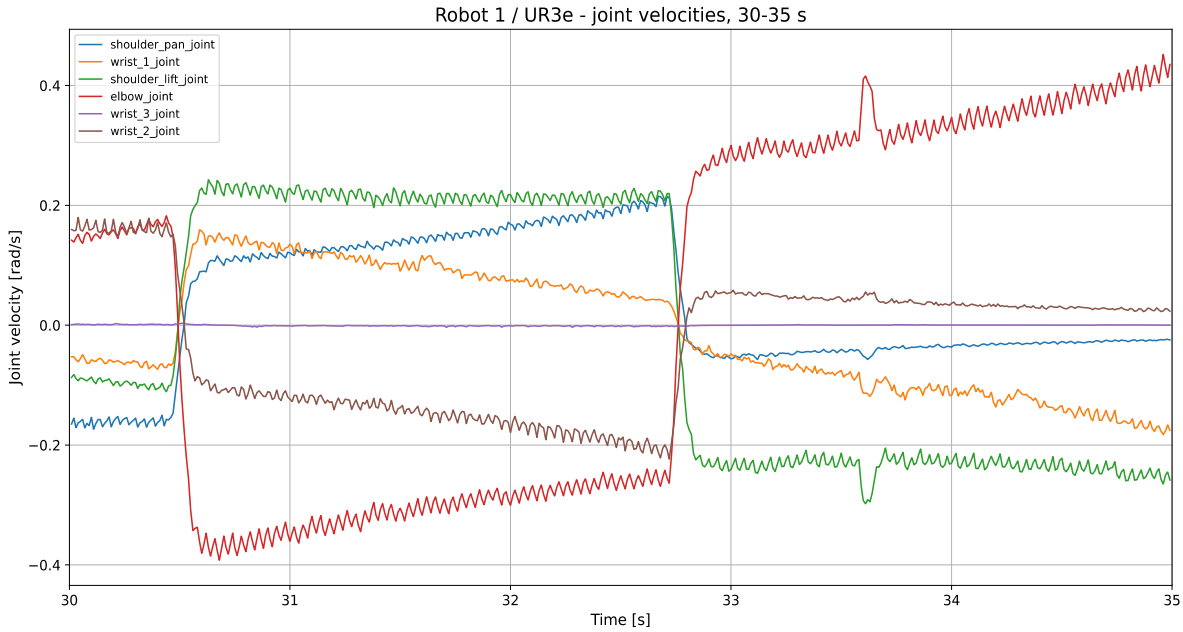
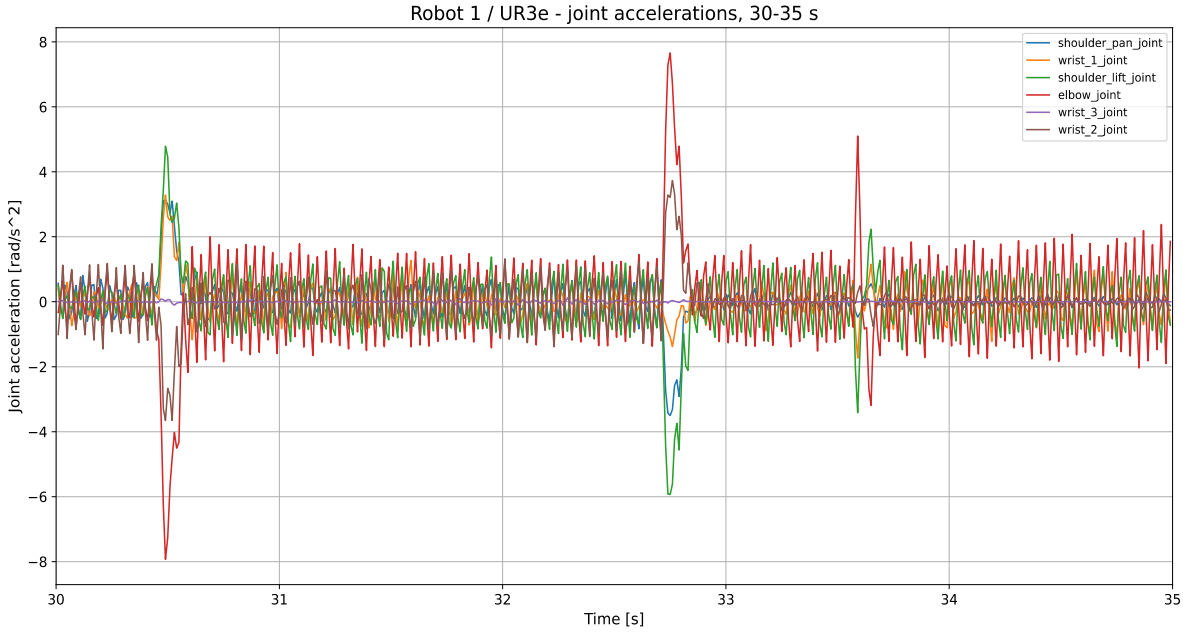


Figure 7.3. Recorded forward position execution without real-time toggle: measured joint velocities and accelerations

Figure 7.3 is similar to the live no-real-time case. This confirms that recording did not significantly change the result when the original live stream was already sufficiently dense.


Robot 1 / UR3e - joint velocities, 30-35 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint velocity [rad/s]
Time [s]


Robot 1 / UR3e - joint accelerations, 30-35 s
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint
Joint acceleration [rad/s^2]
Time [s]

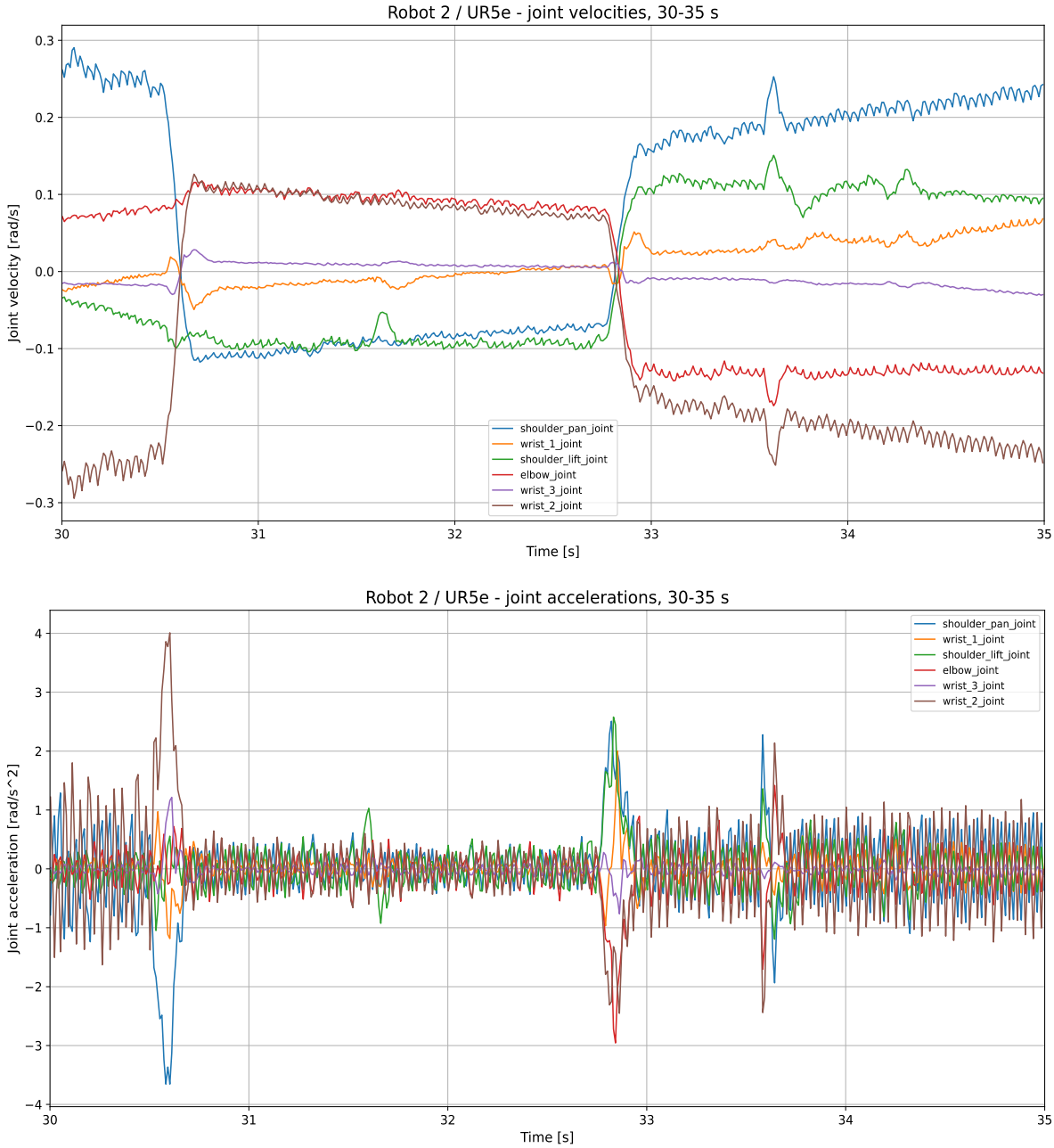


Figure 7.4. Recorded forward position execution with real-time toggle: measured joint velocities and accelerations

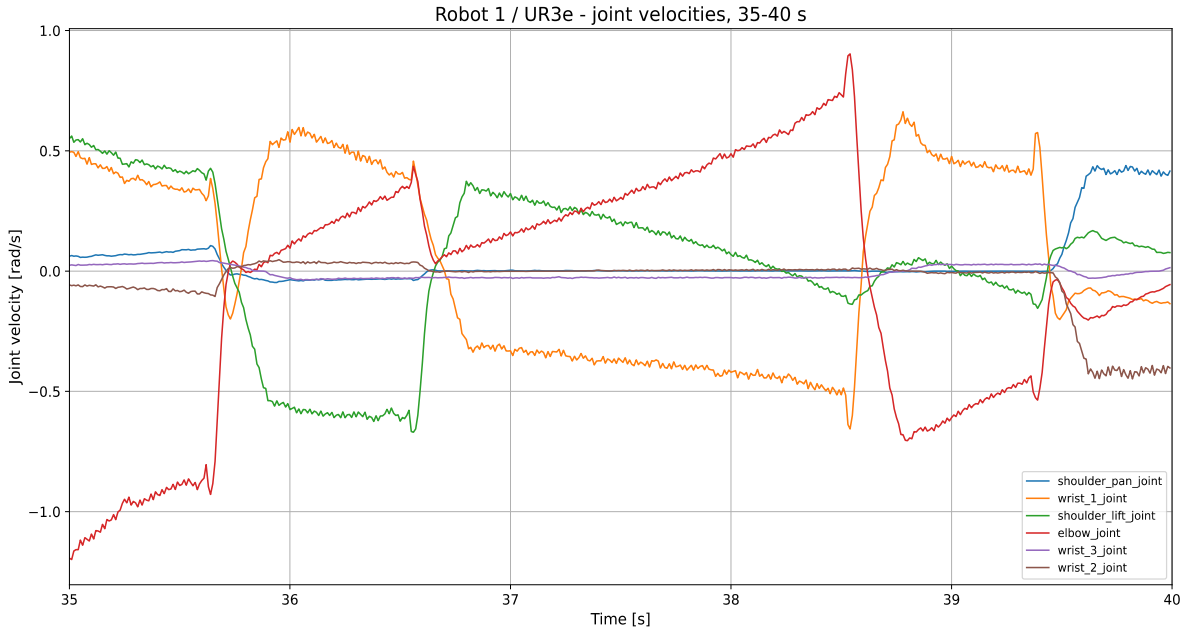
Figure 7.4 shows that the recorded real-time forward position method produced a more regular command execution than the live real-time case. The controller still used only position commands, so slight vibration could still be felt on the robot arm when touched, but the visible pipe vibration was reduced.

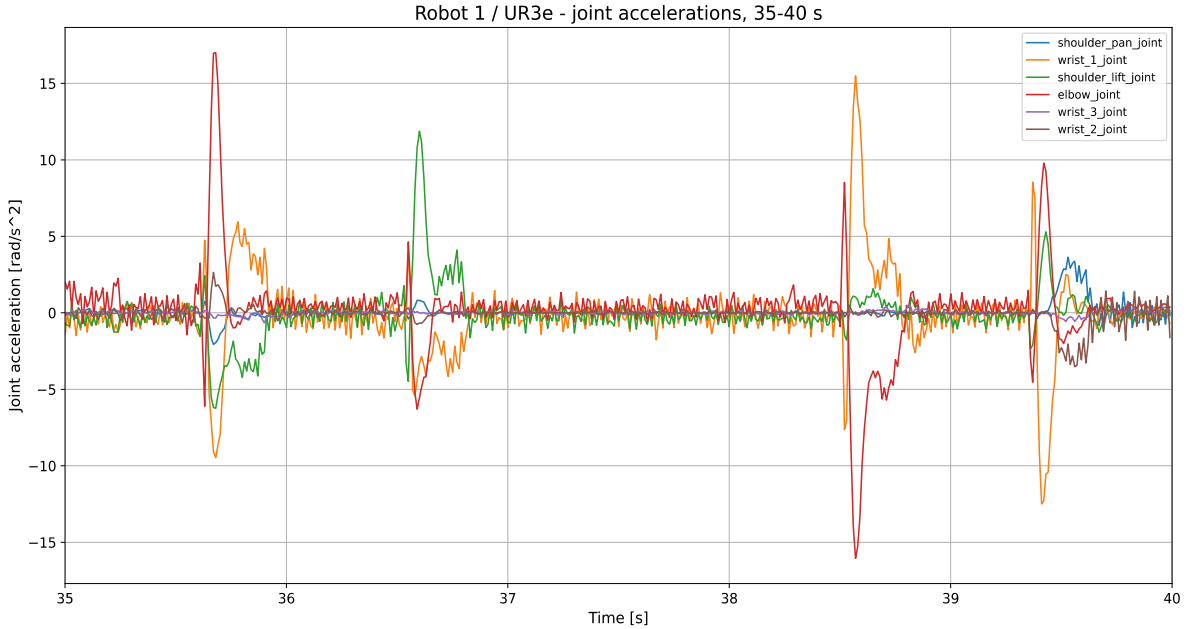
7.7. Recorded JointTrajectory experiments

The recorded JointTrajectory method used the same CoppeliaSim task, but instead of replaying only positions, Python recorded positions, velocities and accelerations. It then generated a complete JointTrajectory for each robot. Every trajectory point contained

positions, velocities, accelerations and a time_from_start value. The whole trajectory was sent to the joint_trajectory_controller before execution.

This is fundamentally different from live position forwarding. The controller does not react to isolated current setpoints. Instead, it receives the complete future path and can interpolate through the trajectory with known timing. This is why both recorded JointTrajectory experiments produced the smoothest physical motion.





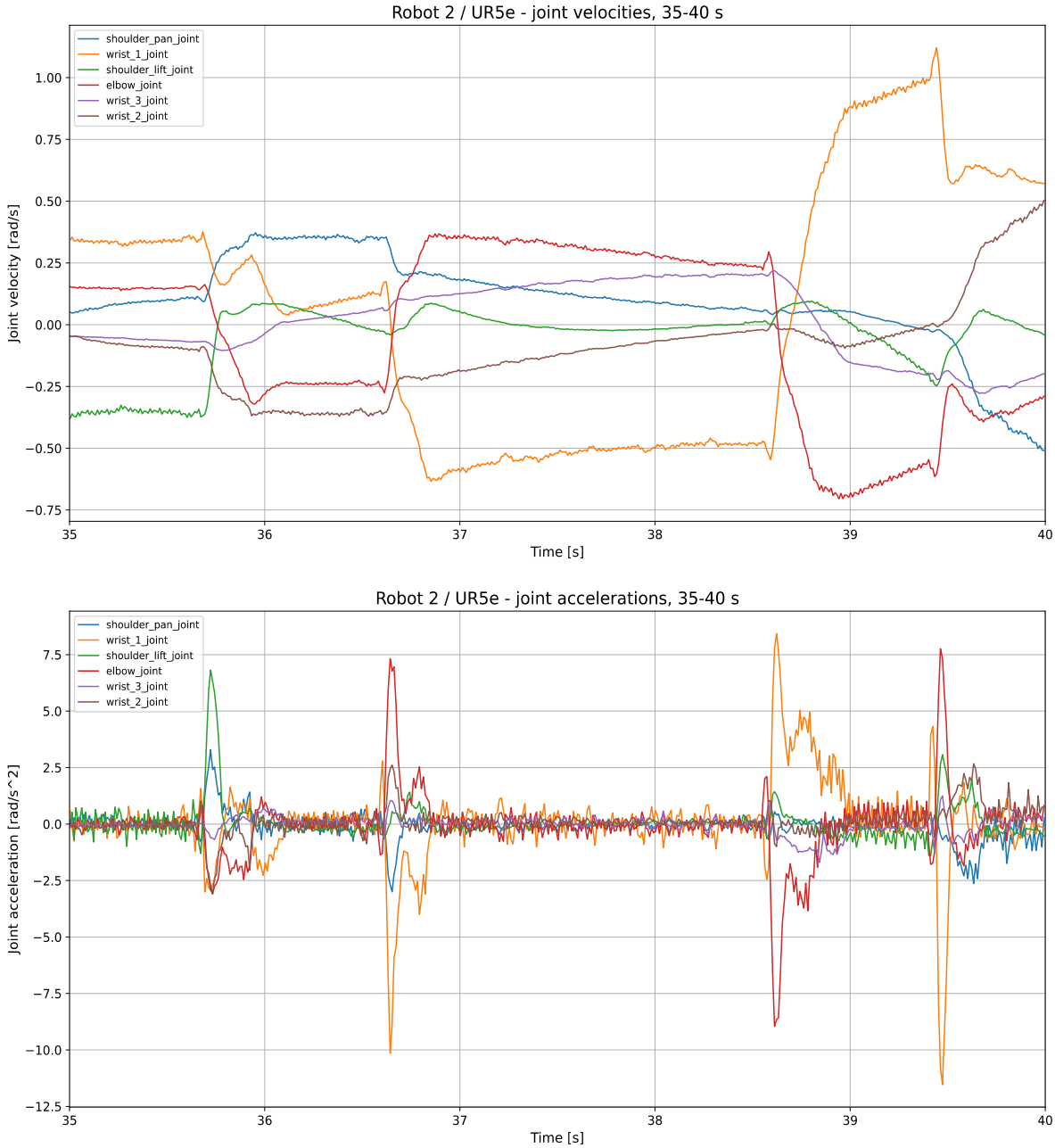
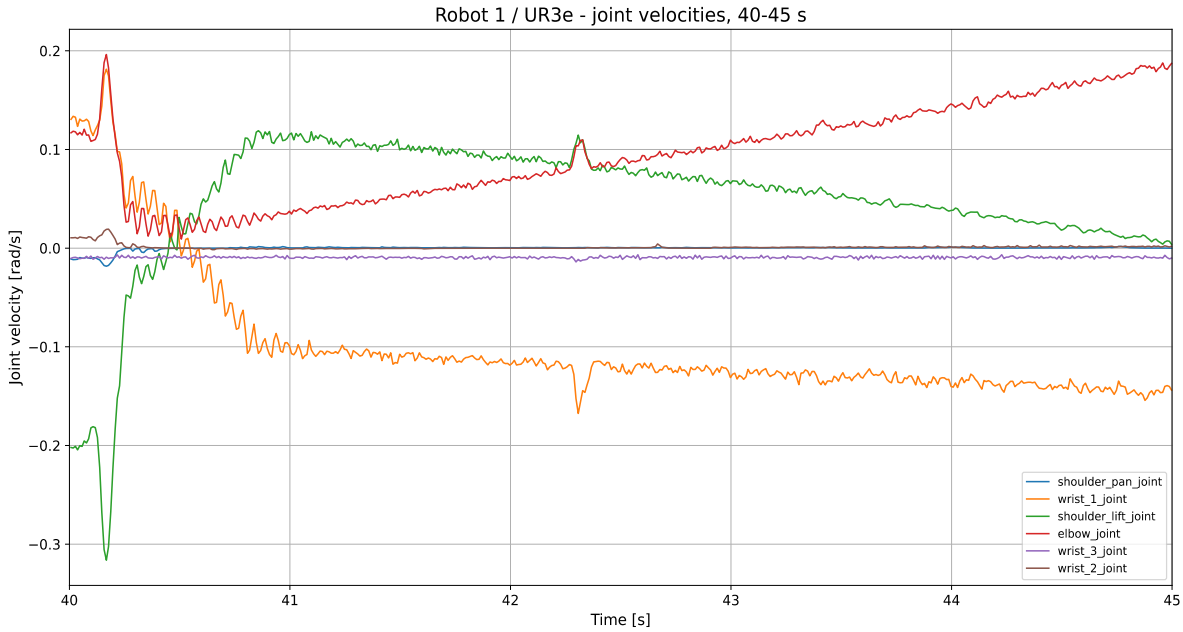
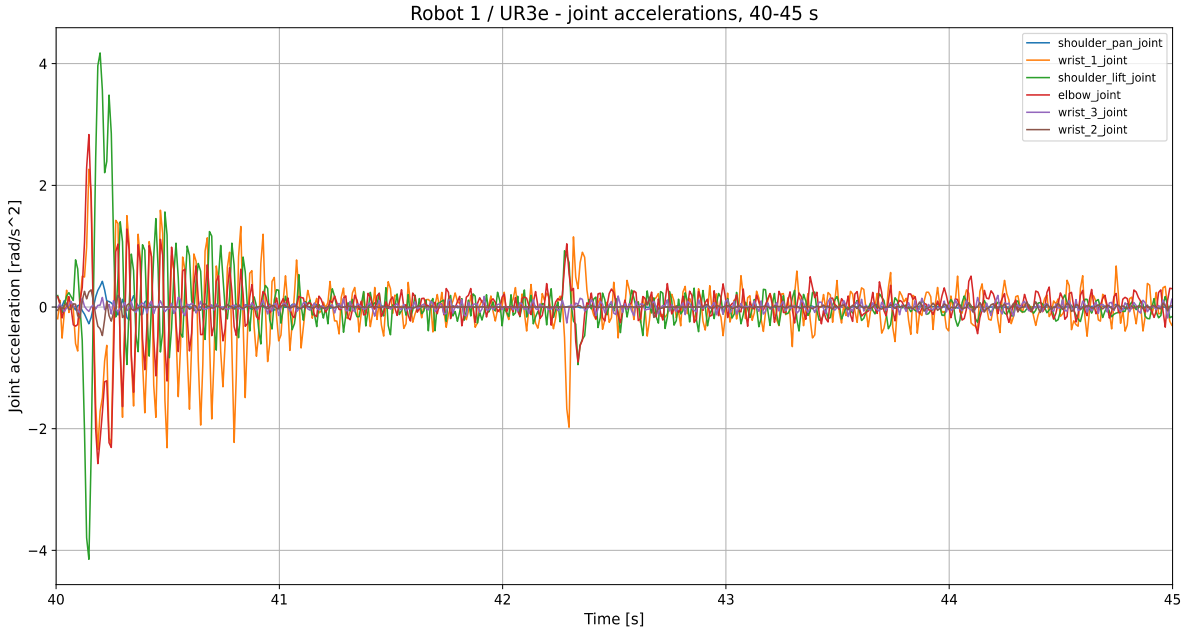


Figure 7.5. Recorded JointTrajectory execution without real-time toggle: measured joint velocities and accelerations

Figure 7.5 corresponds to the no-real-time recorded JointTrajectory experiment. The motion was faster in physical time because the simulation generated the same path in a shorter wall-time interval, but no vibration was observed on the pipe or on the robot arm.


Robot 1 / UR3e - joint velocities, 40-45 s
Joint velocity [rad/s]
Time [s]
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint


Robot 1 / UR3e - joint accelerations, 40-45 s
Joint acceleration [rad/s^2]
Time [s]
shoulder_pan_joint
wrist_1_joint
shoulder_lift_joint
elbow_joint
wrist_3_joint
wrist_2_joint

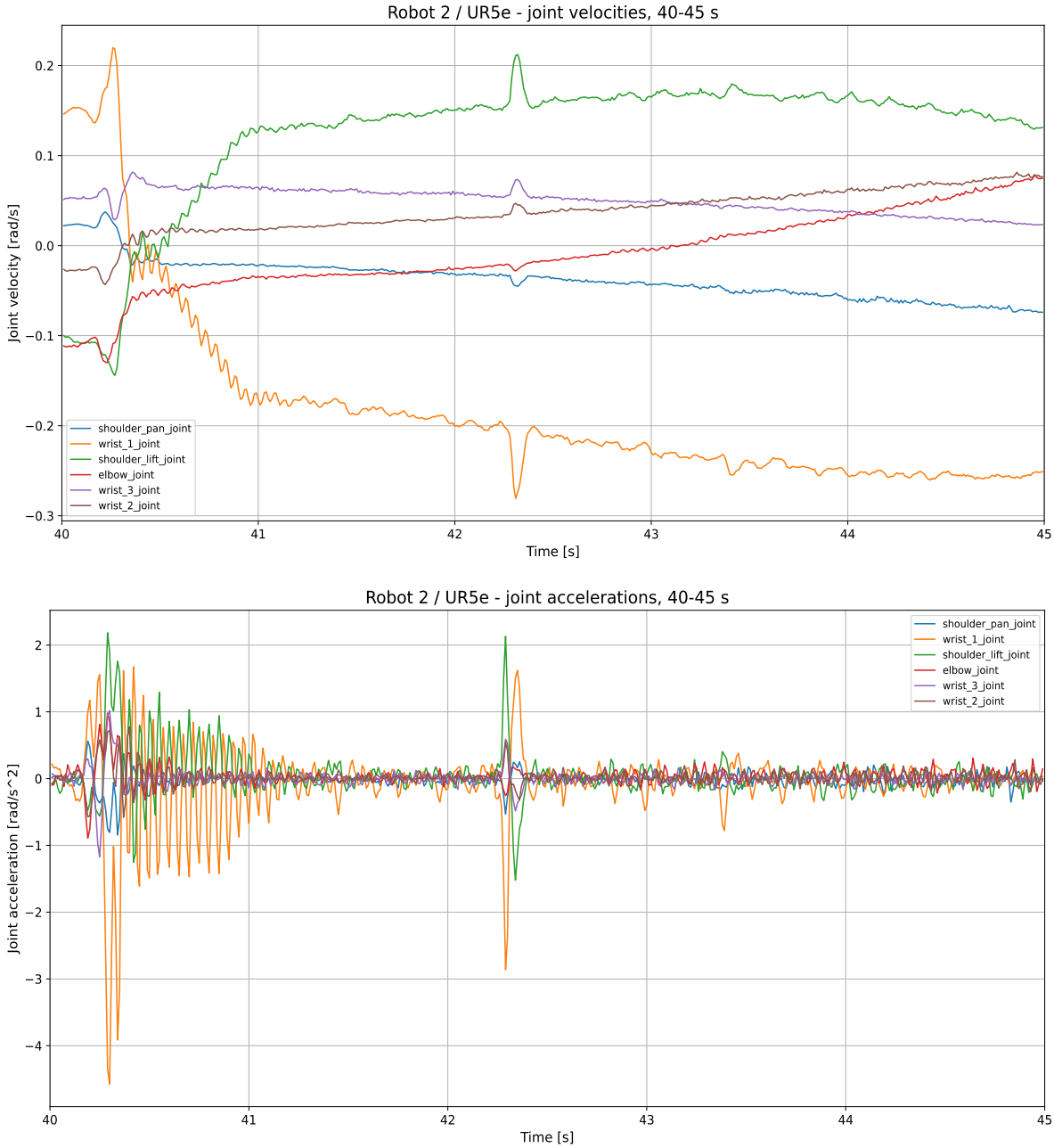


Figure 7.6. Recorded JointTrajectory execution with real-time toggle: measured joint velocities and accelerations

Figure 7.6 corresponds to the real-time recorded JointTrajectory experiment. The motion duration was longer and matched the real-time simulation setting. The important result is that the robot did not vibrate, confirming that the full time-defined trajectory removes the main cause of the vibration observed in the live position method.

7.8. Why live JointTrajectory streaming was not selected

A live JointTrajectory approach was also tested. In that method, CoppeliaSim continuously sent q, q_dot and q_ddot, and Python attempted to build short JointTrajectory commands in real time. This did not produce a stable final method. When FollowJointTrajectory goals were

sent frequently, each new goal could replace the previous one before it was executed. At medium rates the robot jerked, and at higher rates it could fail to follow continuously until the stream stopped.

This result shows that the joint_trajectory_controller is very effective when it receives a complete trajectory, but it is not a suitable high-frequency streaming interface for constantly replacing short trajectory goals in this setup. For this task, the reliable solution was to record the CoppeliaSim motion and send a complete trajectory.

7.9. Overall assessment

The experiments show that vibration is not caused only by the geometric path. It depends strongly on what type of command the controller receives and how the command stream is timed. The forward position controller can work well when the position stream is dense and regular, but it remains sensitive because it receives only positions. Recorded position replay improves the real-time case by removing live Coppelia timing, but it still does not provide velocities or accelerations to the controller.

The recorded JointTrajectory method produced the best result because it included the missing information: positions, velocities, accelerations and time_from_start values. The controller received the entire motion plan in advance, so it could execute the coordinated dual-robot movement without visible pipe vibration.

8. CONCLUSION

This thesis investigated non-sequential task execution with two industrial robots through a gradual development process that combined simulation, motion analysis and real-robot validation. The work started in RoboDK, where point tracking, line following, joint recording and configuration analysis were used to understand the basic task and the limitations of a target-based simulation approach.

The transition to CoppeliaSim made it possible to model the non-sequential character of the task more directly. A moving reference frame was created by placing the path in the structure of one robot, while the second robot followed a moving target using inverse kinematics. The method was then transferred to Universal Robots models and connected to real UR robots through ROS2, the Universal Robots driver and External Control.

The first functional simulation-to-real execution was achieved with the forward position controller. This confirmed that CoppeliaSim joint targets could be transferred to real robots through ROS2. However, additional experiments showed that a position-only controller is sensitive to the density and regularity of the command stream. In the live real-time experiment, the pipe vibrated more because the controller received only discrete position setpoints without velocity, acceleration or future trajectory information.

To investigate this limitation, several recorded and live variants were tested. Recorded forward position replay improved the real-time case because Python reproduced the recorded positions at a regular rate. Nevertheless, the controller still received only positions, so small vibration could remain. This showed that recording alone helps with timing, but does not fully solve the limitation of a position-only command interface.

The best result was achieved with the recorded JointTrajectory method. In this approach, CoppeliaSim was used to generate joint positions, velocities and accelerations. Python recorded these values and created complete time-defined JointTrajectory commands for both robots. Each point contained positions, velocities, accelerations and a time_from_start value. The joint_trajectory_controller therefore received the entire future path before execution.

Both recorded JointTrajectory experiments, with and without the CoppeliaSim real-time toggle, produced smooth physical motion. The pipe did not visibly vibrate and no vibration was felt when touching the robot arm. The real-time toggle affected the physical duration of the recorded motion, but did not introduce vibration because the controller was no longer dependent on live CoppeliaSim setpoint timing.

The final conclusion is that the forward position controller is sufficient for a first functional simulation-to-real transfer, but the recorded JointTrajectory method is the most suitable solution for smooth coordinated execution in this task. The inclusion of joint velocities, joint accelerations and time-defined trajectory points was essential for eliminating the vibration observed in some live position experiments.

The developed system remains an experimental laboratory setup, but it demonstrates a complete workflow from simulation modelling to real dual-robot execution. Future improvements could include automatic collision monitoring, online safety checks, distance-based stopping logic and a more advanced planner capable of generating smooth collision-free trajectories without the need for a separate recording phase.

LITERATURE

[1] RoboDK, RoboDK Documentation - Getting Started, RoboDK, available at: https://robodk.com/doc/en/Getting-Started.html, accessed December 2025.

[2] Coppelia Robotics, CoppeliaSim User Manual - ROS2 Interface, Coppelia Robotics, available at: https://manual.coppeliarobotics.com/en/ros2Interface.htm, accessed February 2026.

[3] Open Robotics, ROS 2 Documentation: Humble Hawksbill, Open Robotics, available at: https://docs.ros.org/en/humble/, accessed April 2026.

[4] Universal Robots, Universal Robots ROS 2 Driver documentation, Universal Robots, available at: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/, accessed April 2026.

[5] Universal Robots, Robot setup and External Control URCap documentation, Universal Robots, available at: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_client_library/doc/setup/robot_setup.html, accessed April 2026.

[6] Universal Robots, Controllers - Universal Robots ROS 2 Driver documentation, Universal Robots, available at: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_robot_driver/ur_robot_driver/doc/usage/controllers.html, accessed April 2026.

[7] ros2_control, forward_command_controller user documentation for ROS 2 Humble, available at: https://control.ros.org/humble/doc/ros2_controllers/forward_command_controller/doc/userdoc.html, accessed June 2026.

[8] ros2_control, joint_trajectory_controller documentation for ROS 2 Humble, available at: https://control.ros.org/humble/doc/ros2_controllers/joint_trajectory_controller/doc/userdoc.html, accessed June 2026.

[9] Open Robotics, trajectory_msgs/msg/JointTrajectory message documentation, available at: https://docs.ros.org/en/humble/p/trajectory_msgs/, accessed June 2026.

[10] Open Robotics, control_msgs/action/FollowJointTrajectory action documentation, available at: https://docs.ros.org/en/humble/p/control_msgs/, accessed June 2026.

[11] Open Robotics, rosbag2 documentation for ROS 2 Humble, available at: https://docs.ros.org/en/humble/Tutorials/Advanced/Recording-A-Bag-From-Your-Own-Node-CPP.html, accessed June 2026.

APPENDICES

Appendix A - Supplementary project repository

The complete supplementary material developed during this thesis is available in the accompanying GitHub repository:

https://github.com/CRTA-Lab/zavrsni_nonsequential_task_execution

The repository contains the complete RoboDK Python scripts, CoppeliaSim Lua scripts and simulation scenes, ROS2 launch and bridge files, recorded-trajectory scripts, data-processing and plotting scripts, and the start-up instructions required to reproduce the developed dual-robot experiments. The thesis contains pseudocode and selected terminal commands only where they are needed to explain the implemented procedures.